

Towards Efficient Privacy-Preserving Machine Learning: A Systematic Review from Protocol, Model, and System Perspectives

WENXUAN ZENG*, TIANSHI XU*, YI CHEN*, and YIFAN ZHOU, Peking University, China

MINGZHE ZHANG, JIN TAN, CHENG HONG, Ant Group, China

MENG LI[†], Peking University, China

Privacy-preserving machine learning (PPML) based on cryptographic protocols has emerged as a promising paradigm to protect user data privacy in cloud-based machine learning services. While it achieves formal privacy protection, PPML often incurs significant efficiency and scalability costs due to orders of magnitude overhead compared to the plaintext counterpart. Therefore, there has been a considerable focus on mitigating the efficiency gap for PPML. In this survey, we provide a comprehensive and systematic review of recent PPML studies with a focus on cross-level optimizations. Specifically, we categorize existing papers into protocol level, model level, and system level, and review progress at each level. We also provide qualitative and quantitative comparisons of existing works with technical insights, based on which we discuss future research directions and highlight the necessity of integrating optimizations across protocol, model, and system levels. We hope this survey can provide an overarching understanding of existing approaches and potentially inspire future breakthroughs in the PPML field. As the field is evolving fast, we also provide a public GitHub repository to continuously track the developments, which is available at <https://github.com/PKU-SEC-Lab/Awesome-PPML-Papers>.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Security and privacy** → **Privacy-preserving protocols**; • **Computing methodologies** → **Artificial intelligence**.

1 Introduction

With the advent of machine learning (ML), artificial intelligence (AI) has ushered in an unprecedented era, profoundly benefiting diverse aspects of society such as smart homes [7, 164], intelligent manufacturing [119, 144], and smart healthcare [68, 170]. In recent years, Transformer-based models [172], especially large language models (LLMs), have emerged as a game-changing revolution in the AI field, such as ChatGPT [145], DeepSeek [74, 126], and Claude [8]. These models demonstrate advanced capabilities in multimodal understanding and complex reasoning [33, 85, 180].

While the models demonstrate exceptional performance, ML as a service (MLaaS) on the cloud has raised serious privacy issues [86, 140, 159, 160]. The clients are required to upload their input prompts to the cloud, which may contain sensitive personal information. Meanwhile, the service provider (i.e., server) like OpenAI is unwilling to offload the trained model to the user in order to protect the proprietary model weights. Hence, despite the convenience and computational power offered by MLaaS, privacy issues remain a critical obstacle to the broader deployment of AI in real-world settings.

To address the issue, privacy-preserving machine learning (PPML) has become a promising and prevalent paradigm for cryptographically strong data privacy protection, fulfilling both parties' requirements¹: the server learns nothing about the user's input, and the user learns nothing about the server's weights, apart from the final inference results as shown in Figure 1(a). PPML includes many research areas such as multi-party computation (MPC), fully homomorphic encryption (FHE), differential privacy (DP), trusted execution environment (TEE), federated learning (FL), etc. In this

* Authors contributed equally.

[†] Corresponding author (meng.li@pku.edu.cn).

¹ This paper primarily focuses on two-party computation (2PC) while computation involving three or more parties will only be briefly discussed.

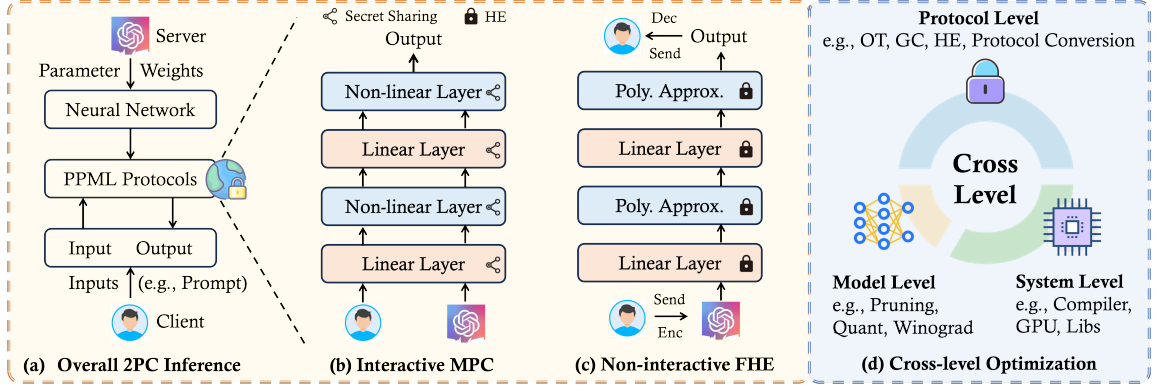


Fig. 1. (a) Overall 2PC inference framework, involving a client and a server. (b) Interactive MPC inference with secret sharing. (c) Non-interactive FHE inference with polynomial approximation for non-linear layers. (d) Paradigm of cross-level optimization for PPML, including protocol, model, and system.

survey, we narrow down the discussed topic to MPC and FHE, and focus on the optimizations across the protocol level, model level, and system level. PPML can be categorized into interactive MPC inference and non-interactive FHE inference as shown in Figure 1. For MPC as shown in Figure 1(b), the client and server jointly perform the linear and non-linear layer computation using the secret sharing (SS) scheme. Commonly, non-linear layers are evaluated using garble circuit (GC) [91, 140, 161] or oblivious transfer (OT) [159, 160, 195] while linear layers are evaluated using OT [159, 160, 195] or homomorphic encryption (HE) [86, 98, 186]. For FHE as shown in Figure 1(c), the client first encrypts the input and then sends it to the server. The server computes the entire model without decryption and then sends the final encrypted output to the client for decryption.

However, the formal privacy protection of PPML comes at a significant cost in terms of efficiency and scalability, as it often incurs orders of magnitude overhead compared to the plaintext counterpart. In the past few years, numerous optimizations have been proposed to achieve better efficiency at the protocol level, model level, and system level. However, there is still a lack of surveys that systematically analyze existing studies and discuss future research directions in the PPML field. In this survey, we highlight three opportunities at the following three levels:

- Opportunity 1 (Protocol Level):** There exists a significant gap between the PPML protocol and plaintext computation. For example, OT brings huge communication overhead, and HE brings huge computational overhead as it operates on ciphertexts representing high-dimensional data but with limited vectorization capabilities (e.g., one-dimensional encoding). Different protocols, such as HE encoding methods, directly affect the overhead. Moreover, existing protocol designs often overlook the inherent characteristics of models, e.g., the robustness to quantization noise and model sparsity. These observations point to the *opportunity for the protocol-level optimization itself and model-aware optimization*.
- Opportunity 2 (Model Level):** Existing model architectures are not friendly to PPML, and model-level optimizations designed for plaintext overlook the characteristics of the underlying protocols. For example, ReLU introduces significant communication cost compared to the plaintext counterpart. However, directly pruning ReLUs ignores the overhead of the linear layer and truncation protocols. Moreover, quantization introduces extra online costs like bit width extension and truncation. Therefore, we highlight the *opportunity for PPML-friendly model architecture design and protocol-aware optimization*.

- **Opportunity 3 (System Level):** Current system-level optimizations are often developed independently of the underlying protocol characteristics, leaving room for tighter integration. For example, many compilers assume fixed encoding domains and lack support for diverse protocol features. Similarly, GPU-based methods typically target general-purpose acceleration but overlook workload-specific factors such as batch size sensitivity and evaluation key usage. These observations point to a valuable *opportunity for protocol-aware system design, which can significantly improve the efficiency and scalability of PPML deployments.*

Considering these three opportunities, in this survey, we comprehensively review the PPML studies across the protocol level, model level, and system level as shown in Figure 1(d). In contrast to our survey, the prior PPML survey [124] does not differentiate between protocol-level and model-level optimizations and lacks system-level discussions. [33] and [143] lack discussions about model-level optimizations. None of these surveys bridges the gap across the three levels or explores the relationship, challenges, and opportunities among them. Therefore, we position this survey as the first to systematically review the PPML field across the protocol level, model level, and system level, with a meticulous PPML taxonomy. To summarize, we make the following contributions in this survey:

- We conduct a comprehensive survey across three levels: **1) protocol level:** we primarily review OT-based and HE-based protocols, identifying the most suitable protocols and HE encoding methods for different scenarios. **2) model level:** we review the studies of linear layer, non-linear layer, and quantization, highlighting the importance of simultaneously optimizing all layers and also emphasizing the necessity of protocol-aware model-level optimization; **3) system level:** we review recent advances in compiler and GPU optimizations, emphasizing the need for PPML-friendly system designs and clarifying how system components can align with protocol characteristics to improve efficiency and scalability.
- We provide qualitative and quantitative comparisons of existing works with technical insights at the protocol level, model level, and system level. Moreover, we provide technical guidelines and discuss the promising avenues for future research, including the *necessity of integrating optimizations across the protocol level, model level, and system level.*
- To continuously track the developments in the PPML field, we also provide a public GitHub repository, which is available at <https://github.com/PKU-SEC-Lab/Awesome-PPML-Papers>.

Taxonomy and Paper Organization. The taxonomy of PPML is shown in Figure 2. We categorize PPML into protocol-level optimization (Section 3), model-level optimization (Section 4), and system-level optimization (Section 5). We divide the protocol-level optimization into linear layers (Section 3.1), non-linear layers (Section 3.2), and computation graph (Section 3.3). At the model level, we focus on PPML-friendly model architecture design for linear layer (Section 4.1), non-linear layer (Section 4.2), and quantization designed for OT and HE protocols (Section 4.3). We discuss system-level optimization from the compiler (Section 5.1) and GPU tailored for PPML (Section 5.2). Finally, we provide an in-depth discussion of the key challenges in cross-level optimization and also provide our view of PPML in the era of LLMs.

2 Background and Preliminaries

2.1 Neural Networks in Machine Learning

2.1.1 Convolutional Neural Networks. Convolutional Neural Networks (CNNs) mainly involve convolution layers and non-linear activation layers, including rectified linear unit (ReLU) and pooling layers.

Convolution layer. Convolution layers are the core building blocks of CNN and involve a lot of multiplications. The parameters consist of a set of learnable filters (or kernels). A single channel 2-dimensional convolution between

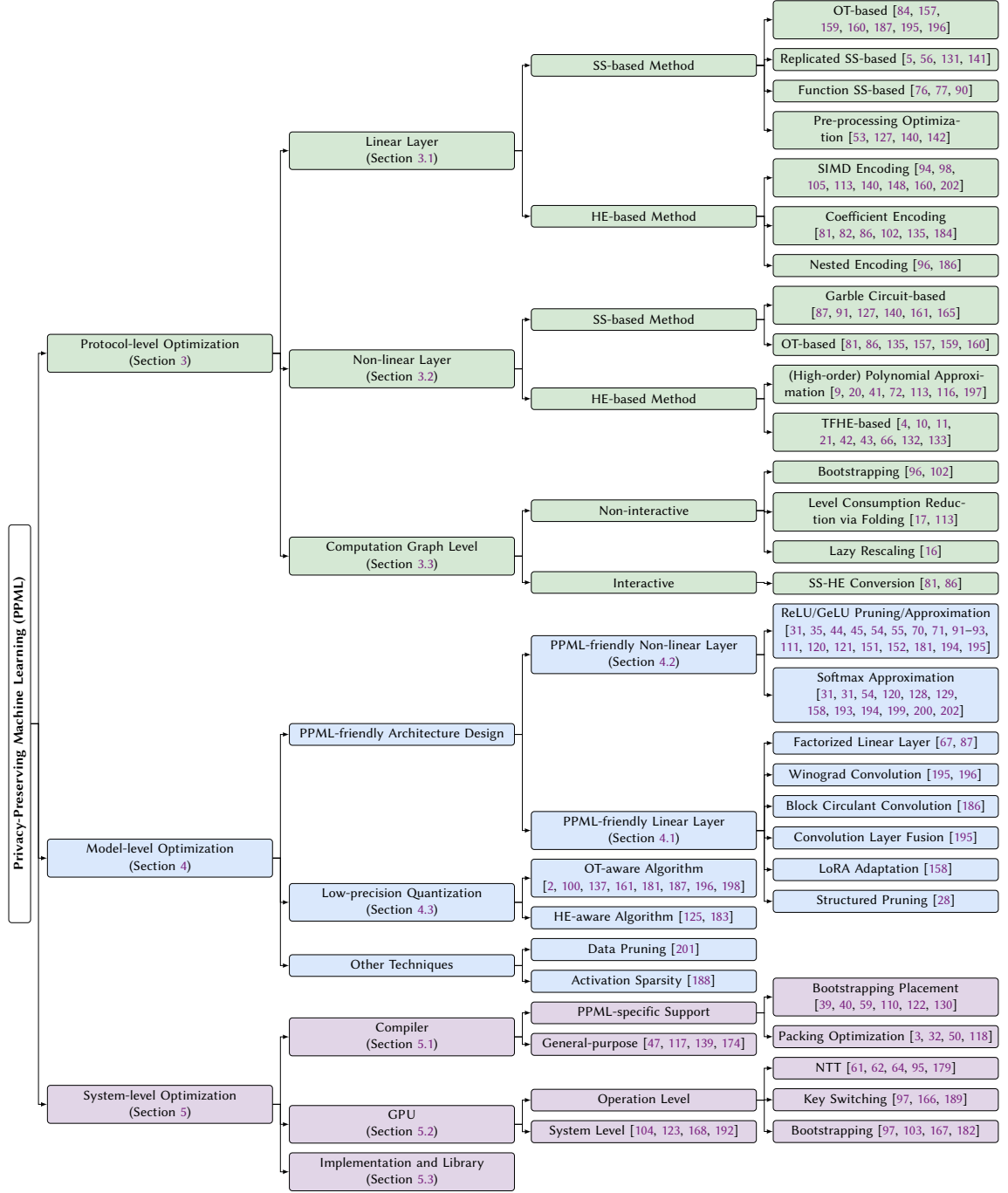


Fig. 2. Taxonomy of existing PPML studies, including protocol-level, model-level, and system-level optimizations.

the filter weights W and the input X can be formulated as

$$\text{Conv}_{i,j} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} W_{m,n} \cdot X_{i+m,j+n}, \quad (1)$$

where M, N are the height and width of the kernel, respectively.

ReLU. ReLU is the most common non-linear layer defined as $\text{ReLU}(x) = \max(0, x)$, which requires element-wise comparison operations and is not cheap in PPML.

Pooling. Pooling layers are used to reduce the shape of feature maps by combining the feature maps into clusters. Max pooling and average pooling are widely used, where the former computes the maximum value per cluster in the feature map and the latter takes the average value.

2.1.2 Transformer-Based Models. The basic Transformer architecture consists of an encoder and a decoder [172]. Vision Transformers (ViTs) follow encoder-only architecture [58] while auto-regressive generative LLMs follow decoder-only architecture [156]. Here, we mainly focus on introducing the decoder-only architecture, and other Transformer-based models share a similar design. An LLM consists of two modules, i.e., attention and feed-forward network (FFN).

Attention. Attention utilizes the dot product to effectively capture the relationship and dependency among the tokens. Attention is defined as

$$\text{Attention}(Q, K, V) = \text{Softmax}(QK^T / \sqrt{d_k}) \cdot V, \quad (2)$$

where d_k is the embedding dimension of the key. Multi-head attention (MHA) is further proposed to guide the model to jointly attend to information from different representation subspaces at different positions, i.e., attention heads.

The generative LLM inference procedure is generally in an autoregressive style such as GPT-2 [156] and LLaMA [171], and mainly consists of two stages: 1) the prefill (prompt) stage and 2) the decoding (generation) stage.

1) Prefill stage. The prefill stage serves as the first step of generation. LLM takes a prompt sequence as input and generates key-value (KV) caches for each layer. The attention can be computed as $\mathbf{O}_{\text{prompt}} = \text{Softmax}(\mathbf{Q}_{\text{prompt}} \cdot \mathbf{K}_{\text{prompt}}^T / \sqrt{d}) \cdot \mathbf{V}_{\text{prompt}}$, where $\mathbf{Q}_{\text{prompt}} \in \mathbb{R}^{H \times T \times d}$ denotes the query and $\mathbf{K}_{\text{prompt}} \in \mathbb{R}^{H \times T \times d}, \mathbf{V}_{\text{prompt}} \in \mathbb{R}^{H \times T \times d}$ denote the key and value cache, respectively. After the prefill stage, the KV cache is generated, which is the foundation for the downstream decoding stage. KV cache retains previously computed key-value pairs, eliminating the need for costly re-computation of previous key and value vectors.

2) Decoding stage. The decoding stage uses and updates the stored KV cache to generate new tokens step by step. Therefore, the attention can be computed as $\mathbf{o}_{\text{dec}} = \text{Softmax}(\mathbf{q}_{\text{dec}} \cdot \mathbf{K}_{\text{cache}}^T / \sqrt{d}) \cdot \mathbf{V}_{\text{cache}}$, where $\mathbf{q}_{\text{dec}} \in \mathbb{R}^{1 \times d}$ denotes the current query. The attention output $\mathbf{o}_{\text{dec}} \in \mathbb{R}^{1 \times d}$ is then sent to the FFN layer for subsequent computation.

FFN. For widely used designs like GPT-2 [156], FFN consists of two linear layers and a Gaussian Error Linear Unit (GeLU) non-linear activation function, which can be formulated as

$$\text{FFN}(x) = W_2(\text{GeLU}(W_1x + b_1)) + b_2, \quad (3)$$

where W_1, W_2 are linear transformations, and b_1, b_2 are two biases. For LLaMA models, FFN is more complex with the SwiGLU activation function and is defined as

$$\text{FFN}(x) = W_2(\text{SiLU}(W_g x) \odot (W_1 x)), \quad (4)$$

where W_1, W_2, W_g are linear transformations, $\odot$ is element-wise multiplication.

2.2 PPML Primitives Involving Two Parties

2.2.1 Garble Circuit. Garbled circuit (GC) is a cryptographic protocol [190] that enables two parties (defined as the Garbler and the Evaluator) to jointly compute a function over the private data. First, the function will be represented as a Boolean circuit. The Garbler garbles the Boolean circuit and generates a garbled table, which is then sent to the Evaluator. The Evaluator receives the garbled table by the OT and evaluates the table to the result.

2.2.2 Secret Sharing. Secret sharing (SS) is the most basic scheme of MPC building blocks over rings. For the commonly used 2-out-of-2 arithmetic secret sharing, the algorithm $\text{Share}(\cdot)$ additively splits an l -bit value x in the integer ring $\mathbb{Z}_{2^l}$ as the sum of two values, denoted by $\langle x \rangle_0$ and $\langle x \rangle_1$. x can be also reconstructed as $\langle x \rangle_0 + \langle x \rangle_1 \bmod 2^l$ by the algorithm $\text{Reconst}(\cdot, \cdot)$. In the 2PC framework, x is secretly shared with the server holding $\langle x \rangle_0$ and client holding $\langle x \rangle_1$. Throughout the entire inference process, intermediate values are secretly shared without any private data exposed.

2.2.3 Oblivious Transfer. Oblivious transfer (OT) is a fundamental cryptographic primitive that has been extensively used in different protocols, including MPC, zero-knowledge proofs (ZKP), and private set intersection (PSI). , with a particular emphasis on PPML. OT allows a sender to input two messages m_0, m_1 and a receiver to input one choice bit $b \in \{0, 1\}$ and then lets the receiver obtain the message m_b based on b . For security, b is kept secret against the sender and the receiver learns nothing about m_{1-b} .

Most PPML protocols first generate Correlated OT (COT) correlations and then transform them to standard OTs in a blazing fast way using a correlation robust hash function (CRHF) [13] and the “pre-computing OT” technique [12]. COT allows a sender to obtain two *random* strings m_0, m_1 where $m_0 \oplus m_1 = \Delta$, and makes a receiver get a *uniform* choice bit $b \in \{0, 1\}$ and the string $m_b = m_0 \oplus b\Delta$, where Δ is fixed for multiple COT correlations.

As OT must involve public-key cryptography (PKC), it usually suffers from poor efficiency due to high computation and communication costs. OT extension (OTE) is hence introduced and allows two parties to extend a small number of PKC-based OT correlations to a large number of OT correlations (even any polynomial number of OTs), thus, significantly reducing computation and communication costs. Up to now, there are three types of OTE: IKNP [89], PCG [23], and PCF [46]. Among them, PCG-style OTE protocols achieve the best performance.

2.2.4 Homomorphic Encryption. Homomorphic encryption (HE) enables computations on encrypted data such that $f(\text{Enc}(x)) = \text{Enc}(f(x))$. In PPML, the most widely used HE schemes are BFV [63], BGV [24], and CKKS [38], all based on the RLWE problem. They support encoding 1D vectors and allow homomorphic addition, polynomial multiplication, element-wise multiplication, and rotation. Except for rotation (single-input), all operations support both ciphertext-ciphertext and ciphertext-plaintext modes. The 1D polynomial multiplication and element-wise multiplication correspond to coefficient encoding and SIMD (Single Instruction, Multiple Data) encoding, respectively. These encoding methods will be explained in detail in Section 3.1.2. Rotation π_i shifts the encoded vector to the right by i positions. For example, $\pi_1([0, 1, 2, 3]) = [3, 0, 1, 2]$. The key differences between BFV/GV and CKKS include: (1) Data type: BFV and BGV only support integer plaintext, whereas CKKS can handle floating-point data. (2) Precision: BFV/GV yield exact results; CKKS introduces approximation errors due to residual noise after decryption. (3) Scale management: CKKS uniquely supports a rescaling operation, which allows homomorphic division by a scalar. This operation can control the scaling factor in fixed-point representations and helps mitigate bit-width growth [185]. BFV/GV do not support this operation, which is why CKKS is the preferred choice for FHE PPML pipelines.

An alternative LWE-based scheme, TFHE [42], supports arbitrary lookup tables via blind rotation, but encodes only a single element per ciphertext, leading to much lower efficiency than RLWE-based schemes.

2.3 Important Features of PPML

2.3.1 Security and Threat Models. One of the most critical components in a PPML framework is the threat model, which fundamentally influences both the design and the security guarantees of the PPML framework. Broadly, threat models can be categorized into the semi-honest threat model and the malicious threat model.

Semi-honest. Most PPML studies adopt the semi-honest threat model (or honest-but-curious) [45, 91, 98, 140, 159, 160, 187, 193, 195, 196]. This means the participating parties are expected to follow the execution requirement of the protocols but also try to find out more information than what they are allowed to find out beyond the final result.

Malicious. The malicious threat model means that participating parties can arbitrarily diverge from the normal execution of the protocol to gain some advantages. Compared with the semi-honest threat model, there are fewer studies focusing on the malicious threat model. For example, ABY3 [141] builds the 3-party computation (3PC) protocols in the malicious threat model scenario. Falcon [176] adopts three servers and assumes an honest majority. FLASH [27], SWIFT [108], Trident [30], and Tetrad [109] provide a malicious threat model with at most one corruption.

Comparison. The malicious threat model is more realistic and is also often more difficult than the semi-honest threat model. In contrast, the semi-honest threat model usually leads to more efficient protocols than the malicious model and, thus, is generally used in the PPML research. Therefore, in this survey, we mainly focus on the PPML studies built on the semi-honest threat model and organize the evolution of these studies.

2.3.2 Interactive and Non-Interactive Protocols. As illustrated in Figure 1, the 2PC framework can be categorized into interactive and non-interactive inference. **Interactive inference (MPC)** requires multiple rounds of communication between the client and the server (or among multiple parties). This pattern requires significant communication due to the frequent data transfer layer by layer. Linear layers are typically computed using HE or OT, while non-linear layers are computed using GC or OT. **Non-interactive inference (FHE)** executes all computations in a single round of communication, with the client pre-processing their inputs and the server computing using HE and returning the final results without any decryption. This pattern requires much less communication than MPC but suffers from much more computation overhead. In Section 3.3, we elaborate graph-level optimizations for both protocols.

2.4 Primary Evaluation Setups in PPML

2.4.1 Network Setups. LAN and WAN are two commonly used network setups for PPML efficiency evaluation. **For LAN setting**, the bandwidth between the machines is 377 MBps and the echo latency is 0.3ms. **For WAN setting**, the bandwidth between the machines is 40 MBps and the echo latency is 80ms.

2.4.2 Evaluation Metrics. In the field of PPML, the studies mainly focus on the evaluation of latency and communication overhead. **Latency** refers to the total time required to complete the inference task with the PPML protocols. LAN and WAN network settings are commonly used for evaluation. **Communication** refers to the data exchange between different parties during the PPML protocol execution.

For example, according to early representative HE-based CryptoNets [72], for a batch input of 4096 MNIST images, it takes 250 seconds to run the model, 44.5 seconds to execute encoding and encryption, and 3 seconds to execute decryption and decoding. It requires around 370 MB of communication. The classical OT-based framework CrypTFlow2 [160] requires 3611.6 seconds (WAN) and 370.84 GB of communication to run a ResNet-50 on ImageNet. The significant inference overhead for different PPML protocols motivates us to explore efficiency optimizations across the protocol level, model level, and system level.

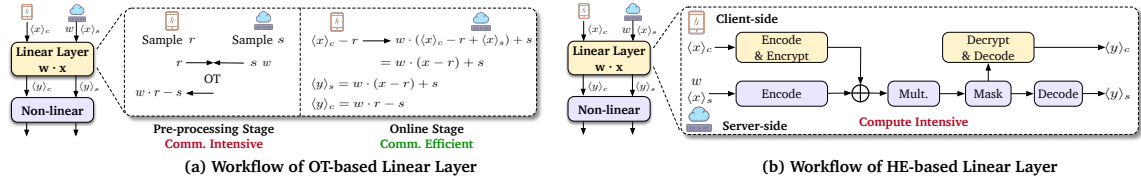


Fig. 3. Basic protocol workflow of (a) OT-based linear layer computation, including a pre-processing stage and an online stage to process client's input; (b) HE-based linear layer computation.

3 Protocol-Level Optimization

3.1 Linear Layer Optimization

Linear layers primarily appear as convolution layers in CNNs, attention layers and FFN layers in Transformers. In this section, we discuss OT-based, HE-based, replicated SS (RSS)-based, and function SS (FSS)-based protocols. Finally, we introduce the pre-processing optimization for efficient online inference.

3.1.1 OT-Based Protocol. We first introduce the basic workflow to leverage OT to compute the linear layers, and then introduce the OT-based protocols.

Workflow. As shown in Figure 3(a), the client holds $(x)_c$ and the server holds $(x)_s$ and weight w where w is not secretly shared in typical PPML scenarios. To generate the result y of a linear layer, the server and client jointly execute computation in the pre-processing (offline) stage and online stage [140]. In the **pre-processing stage**, the client and server first sample random r and s , respectively. Then, $(y)_c = w \cdot r - s$ can be computed with a single OT if $r \in \{0, 1\}$ for 1 bit. With the vector optimization [87], one OT can be extended to compute $w \cdot r - s$, where r and s are both vectors. When w has l_w bits, we can repeat the OT protocol l_w times by computing $w^{(b)} \cdot r - s$ each time, where $w^{(b)}$ denotes the b -th bit of w . The final results can then be acquired by shifting and adding the partial results together. Compared with the pre-processing stage, the **online stage** only requires very little communication to obtain $(y)_s = w \cdot (x - r) + s$.

Cost analysis. Consider a general condition $Y = WX$, where $W \in \mathbb{R}^{M \times L}$, $X \in \mathbb{R}^{L \times N}$ and $Y \in \mathbb{R}^{M \times N}$. With one round of OT, we can compute $W_{i,j}^{(b)} \cdot X_{j,:}$ for the b -th bit of $W_{i,j}$ and j -th row of X . Then, the i -th row of Y , denoted as $Y_{i,:}$, can be computed as $Y_{i,:} = \sum_{j=0}^{L-1} \sum_{b=0}^{l_w-1} 2^b \cdot W_{i,j}^{(b)} \cdot X_{j,:}$, where l_w denotes the bit width of W . Hence, to compute $Y_{i,:}$, in total $O(l_w L)$ OTs are invoked. In each OT, the communication scales proportionally with the vector length of $X_{j,:}$, i.e., $O(N l_x)$, where l_x denotes the bit width of X . The total communication of the GEMM thus becomes $O(M L N l_w l_x)$.

Hence, the total communication of a GEMM scales with both the operands' bit widths, i.e., l_x and l_w , and the number of multiplications, i.e., MLN , both of which impact the round of OTs and the communication per OT. Convolutions follow a similar pattern as GEMM.



Therefore, there are two primary approaches to improve the OT-based linear layer protocol:

- Way 1: Reduce the number of multiplications such as the Winograd convolution algorithm and the circulant convolution algorithm.
- Way 2: Reduce the required bit width of both activations and weights for low-precision inference. We further introduce PPML-friendly quantization algorithms in Section 4.3.

Table 1. HE-based linear layer protocols.

Encoding Type	Literature	Operator	Encoding Consistency	Data Type	Technique
SIMD	Gazelle [98]	Conv, MatMul	✓*	ct-pt	Baseline protocol, Hoisting optimization
	HEAR [105], MPCNN [113]	Conv, MatMul	✓*	ct-pt	Multiplexed format
	Delphi [140]	Conv, MatMul	✓	ct-pt	Pre-processing optimization
	BOLT [148]	MatMul	✓	ct-pt	BSGS optimization, reduce rotations
	E2DM [94]	MatMul	✓	ct-ct	Multiplication of two ciphertext square matrices
	BOLT [148]	MatMul	✗	ct-ct	low multiplication depth ct-ct MatMul for Transformer
	Powerformer [202]	MatMul	✗	ct-ct	Consecutive MatMuls in Transformer
	BLB [185]	MatMul	✗	ct-ct	Consecutive MatMuls in Transformer
	Cheetah [86]	Conv, MatMul	✗	ct-pt	The first coefficient encoding protocols
	Iron [81]	MatMul	✗	ct-pt	Optimized encoding for MatMul
Coefficient	Falcon [184]	Conv	✗	ct-pt	Optimized encoding for MatMul depthwise convolution
	ConvFHE [102]	Conv	✓	ct-pt	Output repacking for batch Conv
	Bumblebee [135]	MatMul	✗	ct-pt	Output repacking to reduce communication
	Rhombus [82]	MatMul	✗	ct-pt	Efficient output repacking
Nested	NeuJeans [96]	Conv	✓	ct-pt	Propose nested encoding and bootstrapping-based fusion
	PrivCirNet [186]	Conv, MatMul	✓	ct-pt	Nested encoding for block circulant Conv and MatMul

* Encoding is inconsistent for Conv with stride > 1.

CryptFlow2 [160] is a classic framework that proposes to use COT to compute the convolution layers, and SiRNN [159] further extends the protocols to lower and mixed bit widths with more complex protocols like bit with extension and wrap. Based on OT-based computation, CoPriv [195] further incorporates Winograd transformation into the convolution protocol, achieving significant multiplication and communication reduction. CipherGPT [84] optimizes the GEMM using VOLE to significantly reduce the communication overhead. PrivQuant [187] proposes the protocol with low precision and EQO [196] further seamlessly combines Winograd convolution and quantization to achieve extreme efficiency improvement.

3.1.2 HE-Based Protocol. The linear layer mainly includes convolutions (Conv) and matrix multiplications (MatMul), which can be efficiently evaluated by RLWE-based HE schemes with minimal communication. These HE schemes operate over polynomials with 1-dimensional coefficient vectors while DNNs compute over tensors. Thus, **Encoding** is required to map a tensor to a vector and directly determines the computational efficiency. There are two types of computation: ciphertext-ciphertext (ct-ct) and ciphertext-plaintext (ct-pt). The former is much more challenging since changing the ciphertext encoding involves heavy HE operations. We begin by categorizing these encoding methods into three categories: **SIMD encoding**, **coefficient encoding**, and **nested encoding**. Table 1 provides an overview of related work. The encoding consistency refers to whether the input and output have the same encoding rule so that we can conduct consecutive Convs and MatMuls without changing encoding algorithms. Below we use (d_1, d_2, d_3) to denote the dimension of a MatMul where $W \in \mathbb{R}^{d_3 \times d_2}$, $X \in \mathbb{R}^{d_2 \times d_1}$ and $Y = W \times X \in \mathbb{R}^{d_3 \times d_1}$. And we use h, w, C, K, R to denote the dimension of a Conv where $X \in \mathbb{R}^{h \times w \times C}$, $W \in \mathbb{R}^{R \times R \times C \times K}$ and $Y = W \otimes X \in \mathbb{R}^{h \times w \times K}$, where $\otimes$ denotes convolution, and we use a default zero padding to ensure the input and output have the same resolution.

SIMD encoding. We begin with SIMD encoding, the first proposed scheme [72] that supports homomorphic element-wise addition and multiplication. Gazelle is a representative work that extends SIMD encoding to both MatMul and Conv. As shown in Figure 4, Gazelle uses diagonal encoding for MatMul, where each diagonal of the weight matrix is encoded into a polynomial, and input rotations $\pi_i(X)$ are applied to align slots. For Conv, multiple rotations and ciphertext-plaintext multiplications (PMult) are performed using different weight polynomials. Gazelle requires

Table 2. Theoretical computation complexity of different works, where n is the number of elements a ciphertext can hold.

Framework	MatMul			Conv		
	# HE-PMult	# HE-Rot	# Ciphertexts	# HE-PMult	# HE-Rot	# Ciphertexts
Gazelle [98]	$O(d_1 d_2 d_3 / n)$	$O(d_1(d_2 + d_3)/n + d_3)$	$O(d_1(d_2 + d_3)/n)$	$O(hwCK/n)$	$O(hw(C + K)/n + K)$	$O(hw(C + K)/n)$
HEAR [105], MPCNN [113]	$O(d_1 d_2 d_3 / n)$	$O(d_1 d_2 d_3 / n)$	$O(d_1(d_2 + d_3)/n)$	$O(hwCKR^2/n)$	$O(hwCKR/n)$	$O(hw(K + C)/n)$
Cheetah [86]	$O(d_1 d_2 d_3 / n)$	0	$O(d_1 d_2 / n + \lceil d_1 / n \rceil d_3)$	$O(hwCK/n)$	0	$O(hwC/n + \lceil hw/n \rceil K)$
Iron [81]	$O(d_1 d_2 d_3 / n)$	0	$O(\sqrt{d_1 d_2 d_3 / n})$	$O(hwCKR^2/n)$	0	$O(\sqrt{hwCKR^2/n})$
Bumblebee [135]	$O(d_1 d_2 d_3 / n)$	$O(d_1 d_3 \log_2 n / (2\sqrt{n}))$	$O(d_1(d_2 + d_3)/n)$	$O(hwCK/n)$	$O(hwK \log_2 n / (2\sqrt{n}))$	$O(hw(C + K)/n)$
Neujeans [96]	$O(d_1 d_2 d_3 / n)$	$O(\sqrt{d_1 d_2 d_3 / n})$	$O(d_1(d_2 + d_3)/n)$	$O(hwCK/n)$	$O(\sqrt{hwCK/n})$	$O(hw(C + K)/n)$
Bolt [148]	$O(d_1 d_2 d_3 / n)$	$O(\sqrt{d_1 d_2 d_3 / n})$	$O(d_1(d_2 + d_3)/n)$	$O(hwCKR^2/n)$	$O(\sqrt{hwCKR^2/n})$	$O(hw(CR^2 + K)/n)$
PrivCirNet [186]	MatMul with circulant weight matrix			Conv with circulant weight kernel		
	$O(d_1 d_2 d_3 / (nb))$	$O(\sqrt{d_1 d_2 d_3 / (nb)})$	$O(d_1(d_2 + d_3)/n)$	$O(hwCK/(nb))$	$O(\sqrt{hwCK/(nb)})$	$O(hw(C + K)/n)$

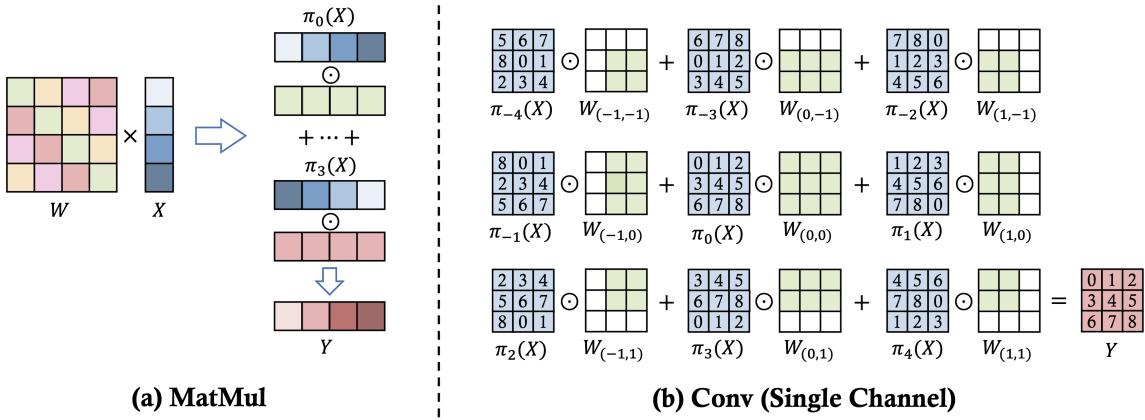


Fig. 4. Toy example of (a) MatMul protocol and (b) Conv protocol using SIMD encoding in Gazelle [98].

$O(d_1 d_2 d_3 / n)$ rotations and PMults for MatMul, and $O(hwCKR^2/n)$ for Conv. It also introduces a hoisting optimization to reduce redundant computation in repeated rotations. Since HE rotations are significantly more expensive than ct-pt multiplications, many subsequent works focus on reducing the number of rotations. Delphi proposes a pre-processing technique where multiplication triples $\langle A \rangle, \langle B \rangle, \langle C \rangle$ with $C = A \times B$ are generated offline as secret shares. This shifts all linear homomorphic computations to the offline phase, requiring only plaintext linear operations online. However, it is only suitable for HE+MPC hybrid frameworks due to the reliance on secret sharing. HEAR [105] introduces a *multiplexed format*, simplifying the rearrangement of convolution outputs, especially when the stride is larger than one, though its basic complexity remains similar to Gazelle. MPCNN [113] leverages the multiplexed format to repack downsampled outputs into a more compact form, improving convolution efficiency.

BOLT [148] improves Gazelle's MatMul protocol by introducing column-packing, which prioritizes the d_3 dimension to reduce slot-wise summation. It further applies Baby-Step Giant-Step (BSGS) optimization to minimize rotation overhead. Theoretically, BOLT achieves $O(\sqrt{d_1 d_2 d_3 / n})$ rotations and $O(d_1 d_2 d_3 / n)$ PMults, with ciphertext I/O reduced to $O(d_1(d_2 + d_3)/n)$ —the current state-of-the-art for ct-pt MatMul.

The above schemes focus on ct-pt operations. For ct-ct settings, encoding becomes more involved. E2DM [94] proposes a ct-ct MatMul protocol for square ciphertext matrices, requiring multiplicative depth 3, with $O(d)$ rotations

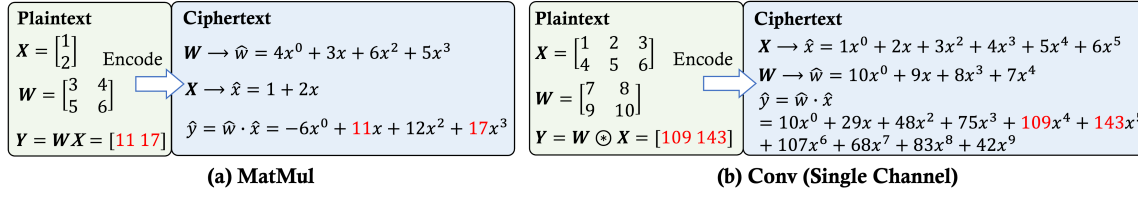


Fig. 5. Toy example of (a) MatMul protocol and (b) Conv protocol using coefficient encoding.

and $O(d)$ ct-ct multiplications. For neural networks like Transformers that involve ct-ct MatMul, BOLT introduces an alternative encoding that reduces the multiplicative depth to 1 at the cost of increasing rotations to $O(d \log d)$, improving overall efficiency since HE cost scales with depth. PowerFormer [149] improves E2DM by optimizing ct-ct MatMul for non-square matrices using replication to reshape them into square form, reducing rotation overhead. However, this method is limited to cases where the matrix aspect ratio is an integer. Building on BOLT [148], BLB [185] introduces further optimizations. It reduces the number of accumulations within a ciphertext via pre-processing and introduces multi-head parallel packing. Encoding different Transformer heads into one ciphertext significantly reduces the rotation cost. Compared to BOLT and PowerFormer, BLB reduces the number of rotations by 29× and 8×, respectively. Although it has a multiplicative depth of 3, the drastically reduced rotation count leads to significantly better overall performance.



Improvements to SIMD Encoding focus on reducing rotations, with two main approaches:

- Way 1: Packing as few elements as possible that need to be accumulated in a ciphertext, e.g., column packing in BOLT [148].
- Way 2: Reuse rotation results as much as possible, e.g., BSGS optimization.

Coefficient encoding. In HE+MPC hybrid frameworks, OT-based protocols are more efficient over the ring $\mathbb{Z}_{2^l}$ than the field $\mathbb{Z}_p$, and modulo reduction over $\mathbb{Z}_{2^l}$ is more hardware-friendly. However, traditional homomorphic SIMD encoding requires computation over $\mathbb{Z}_p$. To better support non-linear protocols on $\mathbb{Z}_{2^l}$, Cheetah [86] implements Conv and MatMul using coefficient encoding by leveraging the similarity between polynomial multiplication and convolution.

Figure 5 shows toy examples of Cheetah’s MatMul and Conv. For MatMul, the input X is split into vectors and encoded as polynomial coefficients, while the weights W are encoded in reverse order. For Conv, X is encoded in row-major order, and the kernel weights are encoded in reverse, with coefficients from different rows spaced by W . Only part of the resulting ciphertext is useful and is extracted as LWE ciphertexts for subsequent 2PC operations. Cheetah achieves $O(d_1 d_2 d_3 / n)$ PMults for MatMul and $O(hwCK/n)$ PMults for Conv, without requiring expensive rotations. Building on Cheetah, Iron [81] introduces an optimization for MatMul by partitioning the (d_1, d_2, d_3) MatMul into smaller blocks of size (d'_1, d'_2, d'_3) such that $d'_1 d'_2 d'_3 \leq n$. This ensures correct computation within each block and reduces transmitted ciphertexts to $\sqrt{\frac{2d_1 d_2 d_3}{n}}$. For visual models like MobileNetV2 and EfficientNet, Falcon [184] optimizes depthwise convolutions by zero-aware greedy packing and communication-aware tiling, reducing overhead caused by padded zero channels.

To improve the packing density, ConvFHE [102] and BumbleBee [135] support packing C RLWE ciphertexts into a single RLWE ciphertext, called PackRLWEs in Rhombus [82]. This requires $O(C)$ homomorphic automorphisms (HomAuto, or slot rotations). Using PackRLWEs, ConvFHE proposes a new encoding that packs multiple convolution

outputs into one ciphertext, reducing I/O overhead. BumbleBee improves PackLWEs by reducing HomAuto overhead, and applies ciphertext interleaving for MatMul, arranging outputs densely via PackRLWEs, reducing transmission to $O(d_1(d_2 + d_3)/n)$ —matching SIMD encoding. Rhombus [82] further reduces PackRLWEs complexity from $O(C)$ to $O(\sqrt{C})$ using a split-point selection technique. It also proposes dimension-aware strategies, choosing between row-major and column-major Conv layouts.



Due to the sparsity of results in ciphertext distribution under coefficient encoding, improvements to coefficient encoding focus on efficiently reducing the number of I/O ciphertexts transmitted. There are two primary approaches to achieve this:

- Way 1: Optimizing the dimensional design for block-wise matrix computation, e.g., Iron [81] and Rhombus [82].
- Way 2: Packing valid coefficients from different ciphertexts into a single ciphertext using HomAuto, while minimizing the number of HomAuto operations, e.g., ConvFHE [102] and BumbleBee [135].

Nested encoding. Neujeans [96] finds that SIMD encoding and Coefficient encoding can be transformed to each other through the discrete Fourier transform (DFT) as shown in Lemma 3.1. The main reason is that polynomial multiplication in the coefficient domain is equivalent to element-wise multiplications in the frequency domain, leading to Lemma 3.1. Based on this, Neujeans implements a nested encoding method for Conv, utilizing coefficient encoding for convolutions of each input channel and SIMD encoding for all input and output channels. This encoding scheme reduces the computational cost to $O(\sqrt{hwCK/n})$ rotations and $O(hwCK/n)$ PMults, while preserving the same input/output consistency, making it the current best encoding algorithm for Conv. PrivCirNet [186] further extends the nested encoding for block circulant Conv and MatMul, reducing the number of rotations to $O(\sqrt{hwCK/nb})$ and the number of PMults to $O(hwCK/nb)$. nested encoding makes use of the advantages of both SIMD and coefficient encoding. However, nested encoding requires DFT to input, which incurs huge computation costs when the input is in the ciphertext domain. To address this, Neujeans proposes fusing convolution with bootstrapping, as bootstrapping inherently involves computing the DFT, allowing the DFT overhead to be amortized and thereby avoiding additional computational cost.

$$\text{LEMMA 3.1. } \langle \text{DFT}(w) \rangle_{\text{SIMD}} \times \langle \text{DFT}(x) \rangle_{\text{SIMD}} = \text{DFT}(\langle w \rangle_{\text{Coeff}} \times \langle x \rangle_{\text{Coeff}})$$

Theoretical complexity. We summarize the computational and communication complexities of existing encoding schemes for ct-pt Conv and MatMul in Table 2. For convolution, the nested encoding scheme achieves the best overall performance, while for matrix multiplication, both BOLT and nested encoding offer optimal efficiency. On the other hand, coefficient encoding is more computation-efficient; however, it suffers from input-output layout mismatches, making it unsuitable for consecutive linear operations and limiting its applicability in practical scenarios.

**Takeaways of different encoding methods:**

- Most SIMD encoding can maintain the consistency of input and output encoding, allowing consecutive computation, and is suitable for FHE scenarios.
- Coefficient encoding eliminates rotation and has lower computational complexity. It has significantly lower computational/communication complexity for convolution than SIMD, but the encoding is inconsistent and is only used for single-layer computation in HE/MPC hybrid protocol scenarios.
- Nested encoding is suitable for parallel processing of multiple convolution scenarios. A single convolution can take advantage of coefficient encoding, and parallel convolution processing can take advantage of SIMD. It is suitable for consecutive computing scenarios, but consecutive calculation requires DFT of the ciphertext, which needs to be integrated with Bootstrapping to reduce overhead.

3.1.3 RSS-Based and FSS-Based Protocol. In this section, we introduce replicated SS (RSS)-based protocols and function secret sharing (FSS)-based protocols.

RSS. 3PC protocols like PUMA [56] and ABY3 [141] are built based on the 2-out-of-3 replicated secret sharing (RSS), where a secret value $x \in 2^\ell$ is shared by three random values $x_0, x_1, x_2 \in 2^\ell$ such that $x = x_0 + x_1 + x_2 \pmod{2^\ell}$, and party P_i gets (x_i, x_{i+1}) (denoted as $[[x]]$).

Let (c_1, c_2, c_3) be public constants and $([[x]], [[y]])$ be two secret-shared values. The secure addition and multiplication procedures are as follows:

- **Addition.** $[[c_1x + c_2y + c_3]]$ can be computed as $(c_1x_0 + c_2y_0 + c_3, c_1x_1 + c_2y_1, c_1x_2 + c_2y_2)$, where P_i can compute its share locally. When $(c_1 = 1, c_2 = 1, c_3 = 0)$, we get $[[x + y]]$.
- **Multiplication.** Parties follow steps: i) first, P_i computes $z_i = x_iy_i + x_{i+1}y_i + x_iy_{i+1}$ locally; ii) parties then perform *re-sharing* by letting P_i sends $z'_i = \alpha_i + z_i$ to P_{i-1} , where $\alpha_0 + \alpha_1 + \alpha_2 = 0$ (P_i can generate α_i using pseudorandom generators with negligible overhead as Mohassel and Rindal [141]); iii) finally, $\{(z'_0, z'_1), (z'_1, z'_2), (z'_2, z'_0)\}$ form the 2-out-of-3 replicated secret shares of $[[xy]]$.

For more details about the 3PC protocol, please refer to Dong et al. [56], Ma et al. [138], Mohassel and Rindal [141].

FSS. FSS has become a trend for private inference where a trusted dealer provides input-independent correlated randomness to the two parties involved in the secure computation, leading to low online costs for 2PC inference. LLAMA [77] proposes FSS-based linear layers such as element-wise multiplication, matrix multiplications, and convolutions. Afterward, Orca [90] and SIGMA [76] follow LLAMA to construct the FSS-based linear layers.

3.1.4 Pre-Processing Optimization. Most of the recent PPML works have adopted the method that divides the inference process into two phases, i.e., a pre-processing phase (offline) and an online phase to transfer the most of the online cost to the offline phase and significantly reduce the online cost [53, 140].

During the pre-processing phase, the core step is to generate input-independent Beaver's triplets, which can be directly consumed by the online phase with much lower communication. If there are only two parties involved in the computation, Beaver's triplets can be generated by two well-known approaches: 1) OT-based generation with lower computation and 2) HE-based generation with lower communication. For instance, MiniONN [127] and Delphi [140] use the HE-based method to generate Beaver's triplets, CoPriv [195] uses the OT-based method to generate Beaver's triplets while ABY [53] and SecureML [142] provide both alternatives for Beaver's triplet generation. It is worth noting

that, as mentioned in CoPriv, the total communication is dominated by the pre-processing phase and cannot be ignored in the PPML inference system.

3.2 Non-Linear Layer Optimization

Non-linear layers primarily appear as ReLU in CNNs, Softmax in attention layers and GeLU/SiLU in FFN layers in Transformers. In this section, we introduce GC-based, OT-based, FSS-based, and HE-based protocols.

3.2.1 SS-Based Protocol. SS-based protocols mainly include GC-based, OT-based, and FSS-based methods.

GC-based protocols. GC is widely used in early PPML research for non-linear layer evaluation. The key feature of GC is that it only requires a constant round to evaluate the function, but suffers from high communication overhead. ABY [53] combines arithmetic sharing, boolean sharing, and GC in one PPML framework. MiniONN [127] uses GC to evaluate ReLU, max pooling, and truncation. Delphi [140] uses GC to evaluate ReLU and a series of works follow Delphi to optimize the model [44, 45, 70, 91, 121, 151]. XONN [161] achieves an end-to-end framework based on GC with a combination of binary neural networks. COINN [87] and ABNN2 [165] choose to evaluate non-linear layers using GC.

OT-based protocols. OT serves as a fundamental building block in cryptographic primitives, capable of realizing any operation in PPML without incurring any approximation errors. All non-linear functions can be implemented based on the basic protocols below:

- **Millionaires' Wrap:** The ℓ -bit Millionaires' functionality, $\mathcal{F}_{\text{Mill}}^\ell$ take $x \in \{0, 1\}^\ell$ from P_0 and $y \in \{0, 1\}^\ell$ from P_1 as input and returns $\langle z \rangle^B$ such that $z = \mathbf{1}\{x < y\}$. The ℓ -bit wrap functionality, $\mathcal{F}_{\text{Wrap}}^\ell$ takes the same inputs returns $\langle z \rangle^B$ such that $z = \mathbf{1}\{x < L - 1 - y\}$.
- **AND:** The functionality $\mathcal{F}_{\text{AND}}$ takes the boolean shares $(\langle x \rangle^B, \langle y \rangle^B)$ as input and returns $\langle x \wedge y \rangle^B$. $\mathcal{F}_{\text{AND}}$ can be realized using bit-triples [160], and the parties can generate two bit-triples through an instance of $\binom{16}{1}$ -OT₂.
- **Boolean to Arithmetic (B2A):** The ℓ -bit B2A functionality, $\mathcal{F}_{\text{B2A}}^\ell$, takes boolean shares $\langle x \rangle^B$ and returns arithmetic shares of the same value $\langle x \rangle^\ell$ by an instance of $\binom{2}{1}$ -OT_ℓ.
- **Multiplexer (MUX):** The ℓ -bit MUX functionality, $\mathcal{F}_{\text{MUX}}^\ell$, takes boolean shares $\langle y \rangle^\ell$ and arithmetic shares and returns $\langle z \rangle^\ell$ such that $z = y$ if $x = 0$ else $z = 0$, which requires two instances of $\binom{2}{1}$ -OT_ℓ.
- **Lookup Table (LUT):** For table T with M n -bits entries, The LUT functionality $\mathcal{F}_{\text{LUT}}^{M,n,T}$ takes $\langle x \rangle^m$ as input and returns $\langle z \rangle^n$ such that $z = T[x]$, which requires an instance of $\binom{M}{1}$ -OT_n.

Through the aforementioned basic protocol, we can implement various non-linear functions such as ReLU [86, 159, 160], GELU [81, 135, 148], and Softmax [81, 135, 148]. Furthermore, truncation and extension can also be achieved [86, 159, 187, 196].

FSS-based protocols. FSS is also widely used in non-linear layers. LLAMA [77] applies FFS-based protocol to various non-linear math functions, including Sigmoid, tanh, and reciprocal square root with precise low bit width representations. Orca [90] proposes FSS-based protocols for CNNs with GPU acceleration. Orca builds basic building blocks, including select and sign extension, and proposes non-linear protocols, including ReLU and Softmax. Different from Orca, SIGMA [76] focuses on Transformers and builds efficient FSS-based building blocks, including truncation, DReLU, and comparison. Based on the building blocks, SIGMA combines them to build complex non-linear layers, including GeLU, Softmax, and layer normalization.

3.2.2 HE-Based Protocol. We introduce two ways to achieve HE-based non-linear protocols, i.e., polynomial approximation and TFHE.

Polynomial approximation. Evaluating non-linear layers using word-wise HE schemes (e.g., BFV, BGV, CKKS) typically relies on polynomial approximation. Low-degree polynomials, pruning techniques, or simple functional replacements can improve inference efficiency but often require model retraining, which we discuss in model-level optimizations in Section 4. When retraining is not feasible, higher-degree polynomials or indirect approximation methods are necessary to meet accuracy requirements—albeit at the cost of increased inference overhead. This section focuses on non-linear function approximation within a pure HE context.

In general, any continuous function can be approximated by polynomial interpolation and evaluated using homomorphic additions and multiplications. For numerical stability, Chebyshev bases are often preferred over power bases. However, discontinuous functions (e.g., sign functions, modular reductions) cannot be approximated directly and instead require indirect methods. The sign function is essential for comparison-based operations such as ReLU, MaxPool, Softmax, and Argmax. An analytic approximation of the sign function using the Fourier series was proposed in [20], while Chialva and Dooks [41] introduced a method based on an approximate tanh function. More recently, minimax polynomial approximations have been widely adopted due to their lower multiplicative depth and reduced use of non-scalar multiplications [112, 114]. Nevertheless, even with such techniques, approximating ReLU with an ℓ_1 -norm error below 2^{-13} in an FHE-based ResNet still requires a multiplicative depth of 14 [113].

TFHE-based protocols. FHE over the Torus (TFHE) was introduced in [42], offering significantly faster sub-second bootstrapping. In TFHE, bootstrapping is programmable, enabling the evaluation of any univariate functions in a look-up table (LUT) format, making it well-suited for nonlinear layer computations. The protocol for ReLU activation was designed in [21], and [43] extended this to MaxPool operations with precision ranging from 8 to 12 bits. Programmable bootstrapping (PBS) in TFHE incurs a computational cost of $O(2^l)$, where l represents the bit width of the input. To address this, [169] and [65] applied the TFHE scheme to quantized neural networks with operands of lower bit width. However, TFHE schemes do not support SIMD operations and suffer from higher latency compared to BFV/CKKS. State-of-the-art TFHE scheme [4] requires 489ms to evaluate a nonlinear function with an 8-bit input, making it challenging to apply in neural network applications due to the high latency. Recent works, such as [10, 11, 132, 133], aim to achieve batch PBS by combining TFHE with BFV and CKKS schemes.



Takeaways of non-linear layer protocols:

- Existing PPML studies use GC-based, OT-based, FSS-based, and HE-based methods to evaluate non-linear functions, and they are tailored to specific application scenarios.
- GC-based protocols only require a constant round but suffer from high communication costs, limiting the adoption in recent works.
- OT-based protocols are general for different non-linear functions but the communication cost is high due to the frequent interactive computation. VOLE can help reduce the communication costs.
- FSS-based protocols provide sublinear communication for specific functions such as equality or comparison, but lack generality.
- HE-based protocols enable low communication costs, but they face high computational overhead when evaluating high-order polynomial approximations and limited support for complex non-linear functions. TFHE leverages PBS to evaluate arbitrary non-linear functions without approximation.

3.3 Graph-Level Technique

Despite linear and non-linear layers, graph-level optimizations are also crucial, focusing on inter-layer techniques for interactive and non-interactive inference.

3.3.1 Interactive Protocols with SS-HE Conversion. Interactive protocols often combine HE with secret sharing, requiring conversion protocols [81, 86]. A typical HE-to-SS conversion works as follows: the server holding $\text{Enc}(x)$ samples a random vector r , computes $\text{Enc}(x - r)$, and sends it to the client. The client decrypts to get $x - r$, while the server keeps r , forming additive shares. The reverse conversion is straightforward: the client encrypts $x - r$ and sends $\text{Enc}(x - r)$ to the server, which adds r homomorphically to recover $\text{Enc}(x)$.

Most protocols follow this pattern. However, BLB [185] recently discovered that applying this to CKKS ciphertexts can lead to security issues [60], due to incorrect treatment of SIMD encoding. They proposed a secure alternative that performs conversion before encoding.

3.3.2 Non-Interactive Protocol. Non-interactive protocols mainly include bootstrapping, level consumption reduction, and lazy rescaling.

Bootstrapping. Unless retraining a HE-friendly network, bootstrapping becomes unavoidable in practical neural networks due to the multiplication depth significantly exceeding the leveled HE parameter configuration. Bootstrapping for CKKS was first introduced in [36] to reset the modulus chain (i.e., the ciphertext level), thereby enabling continued homomorphic computation. Subsequent works [18, 19, 80] proposed optimizations targeting bootstrapping itself.

CKKS bootstrapping comprises several sub-procedures, including *coefficient-to-slots* (CtS), *slots-to-coefficient* (StC), *modular raising*, and *approximate modular reduction*. From a graph-level optimization perspective, these steps may be fused with upstream or downstream components in the neural network computation graph to hide certain computational overhead or avoid redundant domain conversions. For example, ConvFHE [102] leverages domain conversion (i.e., CtS) within bootstrapping to integrate coefficient encoding-based Conv into an end-to-end HE-based protocol, transitioning to the slot domain for element-wise approximate non-linear function evaluation. Similarly, NeuJeans [96] fuses Conv with StC within bootstrapping to conceal convolution overhead, enabling more efficient CNN inference.

Level consumption reduction via folding. To reduce multiplicative depth, nGraph-HE [17] introduces a series of plaintext-ciphertext multiplication folding techniques that fuse scaling operations into adjacent linear layers. These include *AvgPool folding*, which replaces AvgPool–Conv sequences with scaled convolutions; *activation folding*, which transforms polynomial activations into monic forms and propagates scaling factors backward; and *BatchNorm folding*, which absorbs affine normalization parameters into the preceding linear layers. Similarly, MPCNN [113] applies folding in a fully homomorphic ResNet-20 by adapting batch normalization to eliminate the scaling overhead required to fit inputs into $[-1, 1]$ before bootstrapping, thereby avoiding additional multiplicative depth.

Lazy rescaling. nGraph-HE2 [16] proposes *lazy rescaling* to reduce the high cost of rescaling operations in CKKS, which, based on their empirical observations, is significantly more expensive than plaintext-ciphertext multiplication. Instead of rescaling after every multiplication, lazy rescaling defers the operation until the end of a full linear layer, thereby amortizing the cost across multiple multiplications. While effective in practice, this method does not explicitly account for the relationship between the latency and ciphertext level, and can thus be regarded as a heuristic optimization. It may be viewed as a trivial form of scale management, which we discuss more systematically in Section 5.1.3.

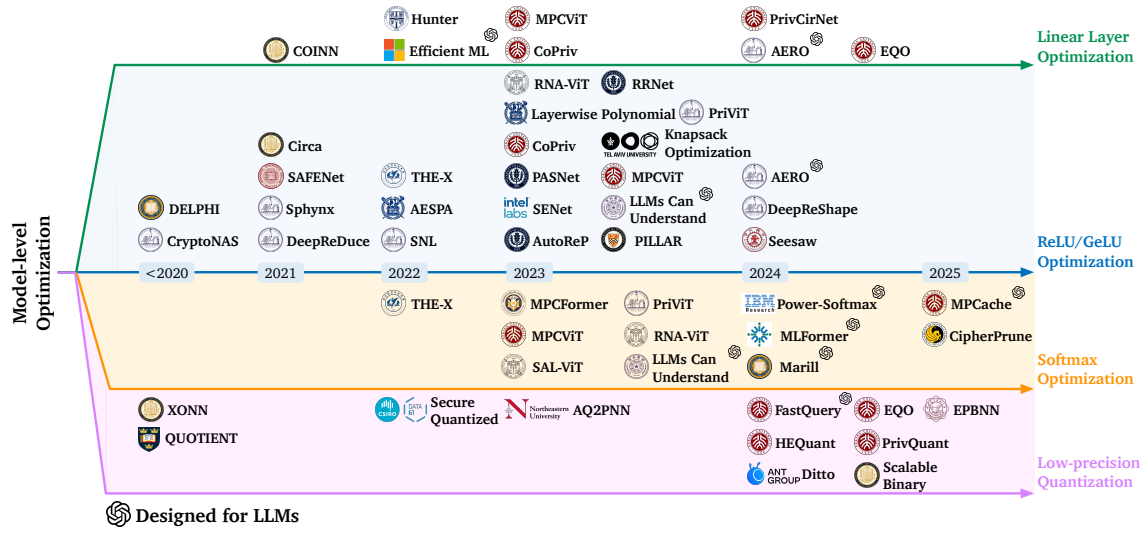


Fig. 6. A timeline of existing model-level PPML optimizations for layer and non-linear layers. Model-level optimization heavily relies on model fine-tuning or re-training.

4 Model-Level Optimization

The model-level optimization mainly focuses on designing a PPML-friendly model architecture, which includes linear layer and non-linear layer optimization. Linear layer is typically convolution and non-linear layer includes Softmax, ReLU, and GeLU. We also discuss quantization methods, which benefit both linear and non-linear layers. Different from protocol-level optimization, model-level optimization typically requires model training or fine-tuning. We summarize the overall model-level optimizations in Figure 6, and mark the methods designed for LLMs that might be very expensive for training or fine-tuning.

4.1 Linear Layer Optimization

Linear layers are ubiquitous in various kinds of neural networks. For CNNs, linear layers mainly involve convolutions, while for Transformers, linear layers mainly involve large matrix multiplications (MatMuls) in attention and feed-forward layers (FFNs). We summarize the studies in Figure 6 (green line) and show the details in Table 3.



Generally, there are two primary ways to improve the efficiency of linear layers:

- Way 1: Reducing the number of multiplications in each MatMul, e.g., Winograd convolution and circulant convolution.
- Way 2: Directly reducing the number of linear layers, e.g., re-parameterization or layer fusion.

For way 1, COINN [87] proposes factorized MatMul by weight clustering to reduce the number of multiplications. Hunter [28] identifies three HE-friendly structures, i.e., internal structure, external structure, and weight diagonal, and three structured weight pruning modules are correspondingly proposed for dot product and convolution. Ganesan et al. [67] replaces regular convolution with factorized convolution like depth-wise convolution and channel-wise shuffle

Table 3. Model-level linear layer optimizations.

Method	Year	Model	Protocol	Backend	# Layers Reduced	# Mult. Per Layer Reduced	Technique
COINN [87]	2021	CNN	SS, OT	EMP-toolkit	✗	✓	Factorized linear layer
Hunter [28]	2022	CNN	HE	-	✗	✓	Structured pruning
Ganesan et al. [67]	2022	CNN	SS, OT	EzPC	✗	✓	Winograd conv., factorized conv.
CoPriv [195]	2023	CNN	SS, OT	EzPC	✓	✓	Winograd conv., re-parameterization
MPCViT [194]	2023	ViT	SS, OT	Secretflow	✓	✗	Layer fusion
PrivCirNet [186]	2024	CNN	HE	SEAL	✗	✓	Block circulant conv.
AERO [92]	2024	GPT-2, Pythia	-	Secretflow	✓	✗	Layer fusion
EQO [196]	2024	CNN	SS, OT	EzPC	✗	✓	Quantized Winograd conv.

to enable information flow across channels. Ganesan et al. [67] also proposes factorized pointwise convolution and uses Winograd convolution for secure inference. PrivCirNet [186] builds a block circulant convolution to simplify convolution computation and further proposes a latency-aware search algorithm for layer-wise block size determination.

For way 2, MPCViT [194] uses NAS to prune the unimportant GeLU in FFN and then fuse two linear layers. AERO [92] also fuses two linear layers in FFN when the ReLU is removed. More comprehensively, CoPriv [195] simultaneously reduces the number of linear layers and multiplications in each layer. Specifically, CoPriv involves Winograd convolution and further proposes re-parameterization to fuse adjacent convolution layers in one block after ReLU pruning. As an optimization of CoPriv, EQO [196] carefully combines Winograd convolution and quantization to further reduce the cost of convolution. We believe that optimizing both things simultaneously can bring effective overall efficiency improvement.

4.2 Non-Linear Layer Optimization

4.2.1 ReLU Pruning and Approximation. ReLU is typically used between two convolution layers in CNNs and defined as

$$y = \max(0, x). \quad (5)$$

However, the max operation is very expensive in PPML. DeepReDuce [91] proposes a three-step method to manually prune redundant ReLUs, which can take a lot of effort and time for optimization. Circa [70] reformulates ReLU as an approximate sign test and proposes a truncation method for the sign test that significantly reduces the cost per ReLU. AESPA [151] uses the low-degree polynomial activation function that exploits the Hermite expansion of ReLU and basis-wise normalization. Gorski et al. [73] finds that ReLU outcomes of nearby pixels are highly correlated, and then proposes to use a single ReLU for each patch. To determine the optimal patch sizes for all layers, this work uses the Knapsack-based optimization strategy to solve this problem. Lee et al. [115] proposes a layer-wise approximation method to optimize the polynomial degree for each layer using the dynamic programming algorithm. PILLAR [55] replaces all ReLUs with a polynomial approximation and proposes polynomial activation regularization to mitigate the accuracy degradation. DeepReShape [91] finds that strategically allocating channels to position the network’s ReLUs based on their criticality to accuracy can simultaneously optimize ReLU and FLOPs efficiency. Therefore, this work proposes ReLU-equalization and ReLU-reuse to enable efficient private inference.

The above works mainly uniformly replace ReLU with other PPML-friendly functions for the whole model. Another line of research focuses on more fine-grained optimizations such as layer-wise, block-wise, and pixel-wise replacement.

These studies utilize various NAS algorithms to search for the redundant ReLUs and prune these ReLUs at different granularities [44, 45, 71, 111, 121, 134, 140, 152–154, 195]. SNL [45] proposes a differentiable NAS method to learn the unimportant ReLUs as defined below:

$$\alpha \cdot \text{ReLU}(x) + (1 - \alpha) \cdot x, \quad (6)$$

where $\alpha \in [0, 1]$ is the architecture parameter assigned to each ReLU at pixel-wise or channel-wise granularity. SENet [111] uses the metric of layer sensitivity to allocate different ReLU budgets for different layers, and then identifies the important ReLUs. CoPriv [195] follows the idea of differentiable NAS in SNL and further proposes a communication-aware NAS algorithm to identify the unimportant ReLUs.

4.2.2 GeLU Pruning and Approximation. GeLU is widely used in FFNs in Transformers like GPT [26] and is defined as

$$\text{GeLU}(x) = \frac{x}{2} \left(1 + \text{erf} \left(\frac{x}{\sqrt{2}} \right) \right), \quad (7)$$

where $\text{erf}(\cdot)$ denotes the Gaussian error function. GeLU is usually approximated with tanh function as

$$\text{GeLU}(x) = \frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right). \quad (8)$$

As observed, GeLU is inefficient for PPML due to the non-linear error function, i.e., tanh and the square root.



Generally, there are two primary ways to reduce the overhead of GeLU:

- Way 1: Replacing GeLU with efficient alternatives such as ReLU and quadratic function through model training.
- Way 2: Approximating GeLU with high-order polynomials without training.

Some works simply replace GeLU with ReLU to reduce the overhead [34, 129], and RNA-ViT [31] replaces GeLU with LeakyReLU to improve the model performance. AERO [92] even more directly prunes the deeper layers due to the observation of their redundancy. MPCFormer [120] and Ditto [181] approximate GeLU with a quadratic polynomial as

$$\text{GeLU}(x) = 0.125x^2 + 0.25x + 0.5. \quad (9)$$

MPCViT [194] and PriViT [54] propose to use differentiable NAS to prune unimportant GeLU. MPCViT further proposes to fuse two adjacent linear layers after pruning to further improve the efficiency. The NAS algorithm is formulated as

$$\alpha \cdot \text{GeLU}(x) + (1 - \alpha) \cdot x, \quad (10)$$

where $\alpha \in [0, 1]$ is the architecture parameter assigned to each GeLU, indicating the importance of each GeLU.

High-order polynomial approximation without training. Different from the above approximations which require training, high-order polynomial approximations can be directly applied to inference; however, introduce a larger overhead to compute the high-order polynomials. PUMA [56] observes that GeLU is almost linear on the two sides, i.e., $\text{GeLU}(x) \approx 0$ for $x < -4$ and $\text{GeLU}(x) \approx x$ for $x > 3$, and proposes a piece-wise polynomial approximation

with four segments as follows:

$$\text{GeLU}(x) = \begin{cases} 0, & \text{if } x < -4 \\ F_0(x), & \text{if } -4 \leq x < -1.95 \\ F_1(x), & \text{if } -1.95 \leq x \leq 3 \\ x, & \text{if } x > 3 \end{cases}, \quad (11)$$

where $F_0(x)$ and $F_1(x)$ are 3-order and 6-order polynomials, respectively computed by the `numpy.polyfit` tool. BumbleBee [135] proposes a similar approximation method as follows:

$$\text{GeLU}(x) = \begin{cases} -\epsilon, & \text{if } x < -5 \\ F_0(x), & \text{if } -5 \leq x < -1.97 \\ F_1(x), & \text{if } -1.97 \leq x \leq 3 \\ x - \epsilon, & \text{if } x > 3 \end{cases}, \quad (12)$$

where ϵ denotes a small value such as $1e-5$, $F_0(x)$ and $F_1(x)$ are 3-order and 6-order polynomials, respectively computed by the `numpy.polyfit` tool. Besides the linearity of GeLU on the two sides, BOLT [148] observes the symmetry of $x \cdot \text{erf}(x/\sqrt{2})$ and proposes to approximate GeLU for half of the input range, i.e., positive input values as follows:

$$\text{GeLU}(x) = \begin{cases} x, & \text{if } x > 2.7 \\ a|x|^4 + b|x|^3 + c|x|^2 + d|x| + e + 0.5x, & \text{if } |x| \leq 2.7 \\ 0, & \text{if } x < -2.7 \end{cases}, \quad (13)$$

where the coefficients a, b, c, d, e are computed with the Remez method.

We summarize the studies of ReLU and GeLU optimizations in Table 4.

4.2.3 Softmax Approximation. For Transformers, the non-linear function Softmax is a key component for attention normalization, which is formulated as

$$\text{Softmax}(x_i) = \frac{e^{x_i - x_{\max}}}{\sum_{j=1}^N e^{x_j - x_{\max}}}, \quad (14)$$

where N denotes the number of tokens and $x_{\max}$ denotes the maximum value of the attention weight for the sake of numerical stability [178].

As can be observed, computing Softmax involves different PPML-inefficient operators, including exponential, max, and division. In recent years, studies have mainly focused on getting rid of these unfriendly operators as much as possible while maintaining the model performance.



Generally, there are three primary ways to reduce the overhead of Softmax:

- Way 1: Replacing PPML-inefficient operators, including exponential, max, and division with efficient operators through model training.
- Way 2: Directly reducing the number of Softmax, e.g., head pruning/merging and KV cache compression.
- Way 3: Replacing PPML-inefficient exponential with high-order polynomials without training.

Most of the works focus on way 1 by removing the inefficient operators. MPCFormer [120] and SecFormer [136] uniformly replaces exponential with quadratic function as $\text{Softmax}(x) \approx (x + c)^2 / \sum (x + c)^2$ where c is a constant. MPCViT [194] proposes to replace exponential with ReLU and further use scaling attention for “unimportant” heads using latency-aware neural architecture search (NAS). Following MPCViT, PriViT [54] directly replaces the whole

Table 4. Model-level non-linear ReLU and GeLU optimizations.

Method	Year	Model	Protocol	Backend	NAS	Granularity	Technique
Delphi [140]	2020	CNN	GC	fancy-garbling	✓	Layer	Population algorithm
CryptoNAS [71]	2020	CNN	GC	MiniONN	✓	Layer	Macro-search and ReLU shuffling
DeepReDuce [91]	2021	CNN	GC	Delphi	✗	Layer	Manually pruning
SAFENet [134]	2021	CNN	GC	fancy-garbling	✓	Channel	Multiple-degree Poly. approximation
Sphynx [44]	2021	CNN	GC	Delphi	✓	Block	Micro-search algorithm
Circa [70]	2021	CNN	GC	Delphi	✗	Model	Piece-wise linear and sign function
SNL [45]	2022	CNN	GC	Delphi	✓	Pixel, Channel	Differentiable search
AESPA [151]	2022	CNN	GC	Delphi	✗	Model	Hermite ReLU expansion
THE-X [34]	2022	Bert	-	-	✗	Model	Replace GeLU with ReLU
RRNet [153]	2023	CNN	SS, OT	-	✓	Layer	Hardware-aware search
MPCFormer [120]	2023	Bert, Roberta	SS, OT	CrypTen	✗	Model	Replace GeLU with quadratic poly.
MPCViT [194]	2023	ViT	SS, OT	Secretflow	✓	Layer	Latency-aware search
PriViT [54]	2023	ViT	SS, OT	Secretflow	✓	Layer	Differentiable search
CoPriv [195]	2023	CNN	SS, OT	EzPC	✓	Block	Communication-aware search
AutoRep [152]	2023	CNN	SS, OT	CrypTen	✓	Pixel	Distribution-aware poly. approx.
SENet [111]	2023	CNN	GC	Delphi	✓	Layer, pixel	Sensitivity-based pruning
PASNet [154]	2023	CNN	[69]	[69]	✓	Layer	Hardware-aware search
Gorski et al. [73]	2023	CNN	3PC compare	SecureNN	✗	Patch, channel	Knapsack-based pruning
Liu and Liu [129]	2023	Bert, Roberta	SS, OT	EzPC	✗	Model	Replace GeLU with ReLU
Lee et al. [115]	2023	CNN	HE (RNS-CKKS)	Lattigo	✗	Layer	Layer-wise degree poly. with dynamic programming
PILLAR [55]	2023	CNN	SS, OT	CrypTen	✗	Model	Polynomial regularization
RNA-ViT [31]	2023	ViT	SS, OT	CrypTen	✗	Model	Replace GeLU with LeakyReLU
AERO [92]	2024	GPT-2, Pythia	-	Secretflow	✗	Layer	Deeper-layer pruning
Seesaw [121]	2024	CNN	GC	Delphi	✓	Block	Bi-level search
DeepReShape [93]	2024	CNN	GC	Delphi	✗	Stage	ReLU equalization and reuse
Ditto [181]	2024	Bert, GPT-2	RSS	Secretflow	✗	Model	Replace GeLU with quadratic poly.

Softmax with a quadratic function as $\text{Softmax}(x) \approx x^2/N$ where N is the token number, and combines it with original Softmax using NAS. SAL-ViT [199] further proposes to combine self-attention and external attention [75] using NAS to improve MPCViT. RNA-ViT [31] first uses linear layers to reduce the dimension and then uses a high-order polynomial (HOP) to approximate Softmax as $\text{Softmax}(x) = (x + c)^4 / (\sum (x + c)^4 + \epsilon)$ where c is a constant and ϵ is a small positive value to avoid a zero denominator. MLFormer [128] considers this issue from the perspective of linear attention and proposes scaled linear attention to avoid Softmax. Power-Softmax [202] proposes $\text{Softmax}(x) \approx x^p / \sum x^p$ where p is a positive even number. To circumvent the use of vector-wise Softmax, Zimmerman et al. [203] employs an element-wise function like a quadratic function or a high-degree polynomial.

For way 2, Marill [158] proposes using head pruning and merging in PPML to directly reduce the number of Softmax, thus reducing the inference overhead. Note that the process of deciding which heads to prune or merge is done offline [15], so there is no additional inference overhead. MPCache [193] for the first time incorporates the KV cache eviction algorithm with PPML-friendly optimizations to drastically reduce the KV cache, thereby lowering the Softmax overhead. CihperPrune [84] proposes encrypted token pruning to remove unimportant tokens layer by layer and assigns lower-degree polynomials to less important tokens after pruning.

High-order polynomial approximation without training. PUMA [56], CipherGPT [84], and BumbleBee [135] approximate exponential using Taylor series as follows:

$$\exp(x) = \begin{cases} 0, & \text{if } x \leq m \\ (1 + \frac{x}{2^t})^{2^t}, & \text{if } m \leq x \leq 0 \end{cases}, \quad (15)$$

Table 5. Model-level non-linear Softmax optimizations.

Method	Year	Model	Protocol	Backend	NAS	Granularity	Technique
THE-X [34]	2022	Bert	HE (CKKS)	SEAL	✗	Model	Extra estimation model
MPCFormer [120]	2023	Bert, Roberta	SS, OT	CrypTen	✗	Model	Replace e^x with x^2
MPCViT [87]	2023	ViT	SS, OT	Secretflow	✓	Head	Replace e^x to ReLU and scaling attention
SAL-ViT [199]	2023	ViT	SS, OT	CrypTen	✓	Head	Replace self attention with external attention
PriViT [54]	2023	ViT	SS, OT	Secretflow	✓	Row	Replace Softmax with Square
RNA-ViT [31]	2023	ViT	SS, OT	CrypTen	✗	Model	Higher-order poly. approx.
MLFormer [128]	2024	ViT	SS, OT	CrypTen	✗	Model	Scaled linear attention
Marill [158]	2024	LLaMA, Deepseek Coder	GC	SecureML	✗	Head	Head pruning and merging
Power-Softmax [202]	2024	Pythia, Roberta	HE (CKKS)	HEaaN	✗	Model	Poly. approx. and Lipschitz division
Zimmerman et al. [203]	2024	ViT, Bert	HE (CKKS)	HEaaN	✗	Model	HE-friendly poly. approx.
SecFormer [136]	2024	Bert	SS, OT	CrypTen	✗	Model	Replace e^x with x^2
MPCache [193]	2025	GPT, LLaMA	SS, OT	Secretflow	✗	Head	KV cache compression
CipherPrune [200]	2025	GPT, Bert	SS, OT, HE	EzPC	✓	Layer	Token pruning and poly. approx.

where m and t are set to different values in these works with different efficiency and performance trade-offs. Specifically, BumbleBee sets $a = -13$, $t = 6$, CipherGPT sets $a = -16$, PUMA sets $a = -14$, $t = 5$.

LUT-based evaluation. Some studies directly utilize LUTs to evaluate the non-linear Softmax, which pre-compute and store the input-output pairs in the table to avoid costly real-time computation. Iron [81] follows CryptFlow2 [160] and SiRNN [159] and uses OT-based LUT to evaluate the exponential and reciprocal. Pika [175] uses large LUTs for exponential and reciprocal, but large LUTs are very inefficient. SIGMA [76] further proposes to minimize the size of LUTs with lower bit widths while maintaining accuracy.



Takeaways of PPML-friendly non-linear layers:

- As observed in these studies, approximation or pruning at model-wise granularity can hurt the model performance significantly. To alleviate this problem, more and more studies focus on fine-grained approximations like layer-wise and head-wise using NAS algorithms.
- Replacing non-linear layers with simpler operators or low-order polynomials requires training to recover the model performance. In contrast, when approximating non-linear layers using higher-order polynomials (≥ 3 -order) does not necessarily require training, but suffers from higher overhead for evaluating the polynomial.

4.3 Low-Precision Quantization

Quantization is a widely used technique that compresses the activations or weights to low bit widths for efficient training or inference. Quantization benefits both linear and non-linear layers. However, directly applying quantization to PPML is a non-trivial practice for both SS-based and HE-based frameworks. We summarize the quantization methods designed for PPML in Table 6.

4.3.1 SS-Friendly Algorithm. XONN [161] is a very early work that attempts to reduce the cost of the GC protocol. XONN uses binary neural networks (BNNs) which quantize the activations and weights to binary values to replace the costly multiplications with XNOR operations. COINN [87] enforces the scale to be powers of 2, which allows

Table 6. Model-level PPML-friendly quantization optimizations.

Method	Year	Model	Protocol	Backend	Mixed-Precision	Technique
XONN [161]	2019	CNN	GC	Customization	✗	Binary quant.
COINN [87]	2021	CNN	SS, OT, GC	EMP-toolkit, TinyGarble2	✓	Genetic algorithm
PrivQuant [187]	2023	CNN	SS, OT	EzPC	✓	Communication-aware quant.
EQO [196]	2024	CNN	SS, OT	EzPC	✓	Communication-aware Winograd quant., bit re-weighting for outliers
Ditto [181]	2024	Bert, GPT-2	RSS	Secretflow	✗	Static dyadic quant.
FastQuery [125]	2024	LLaMA-2	HE	OpenCheetah	✓	Communication-aware embedding table quant.

us to compute the costly scale operation using logical shifts in GC with nearly zero cost. PrivQuant [187] proposes to use mixed-precision quantization based on communication-aware Hessian information [57] for a better trade-off between model accuracy and efficiency. PrivQuant finds that direct quantization even increases the inference overhead due to expensive bit width conversions, including extension and re-quantization. PrivQuant then proposes a series of graph-level optimizations like protocol fusion and most significant bit (MSB)-known optimization to reduce the overhead. EQO [196] further combines efficient Winograd convolution and mix-precision quantization, but observes that Winograd transformations can introduce weight outliers and extra costly bit width extensions. To solve these problems, EQO proposes a bit re-weighting strategy to accommodate the outliers and graph-level protocol fusion to remove the extra extensions. Ditto [181] adopts static dyadic quantization to avoid dynamically computing scale during inference and proposes type conversion protocols for efficient bit width conversion.

4.3.2 HE-Friendly Algorithm. FastQuery [125] features a communication-aware embedding table quantization algorithm that assigns different bit widths for different channels. FastQuery leverages the one-hot nature of queries and the low-precision embedding table to reduce the accumulation bit width. To further reduce the communication for sub-13-bit quantization, FastQuery proposes a novel element packing algorithm to squeeze the embedding table dimension.



Takeaways of PPML-friendly quantization:

- Directly combining quantization and PPML is a non-trivial practice. Instead, the combination usually requires model/algorithm and PPML protocol co-optimization to enable end-to-end efficiency improvement.
- Although there are several SS-friendly quantization algorithms, the algorithm designed for HE is still scarce. We believe the main reason is that quantization is not simple for HE, requiring very refined quantization strategies, modulus selection, encoding methods, etc.

5 System-Level Optimization

System-level optimization is essential for accelerating PPML, as the computational bottleneck typically resides in the HE component, whether used alone or in combination with MPC. Protocol-level encoding optimizations reveal challenges of designing efficient HE-style workflows. While such techniques can reduce the number of homomorphic operations, they offer limited relief from the intrinsic computational overhead of each operation. Compiler technologies help bridge the semantic gap between high-level ML workloads and low-level HE primitives, while GPUs, as the most accessible accelerators, help narrow the performance gap between encrypted and plaintext computation. This section highlights recent advances in HE compilers and GPU optimizations, which together enable scalable and efficient PPML systems.

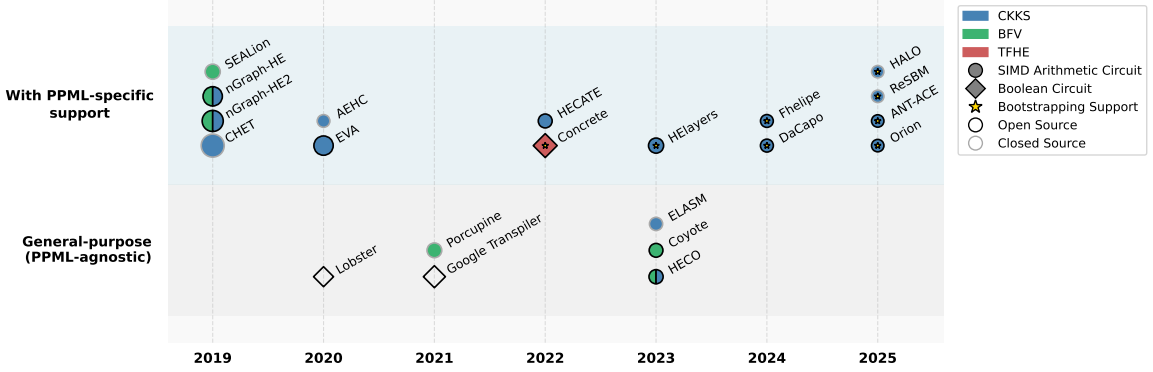


Fig. 7. Timeline of HE Compilers. PPML-specific compilers are positioned in the upper region (Classification is based on whether the corresponding paper introduces an ML-friendly IR or DSL, or includes evaluations on PPML tasks).

5.1 Compiler

We categorize existing HE compilers into two groups based on their target workloads: **general-purpose** compilers and those **with PPML-specific support**, as illustrated in Figure 7. We first motivate the need for HE compilers, especially for PPML, and then review core techniques.

5.1.1 The Need for HE Compiler. Writing efficient FHE programs manually is extremely difficult, due to the mathematical structure and the practical demands of real-world workloads.

At a general level, programming in HE involves several unique challenges:

- (1) **Dataflow constraints:** HE programs must be expressed as static circuits or dataflow graphs, without native control flow constructs like branching or loops.
- (2) **Limited instruction set:** FHE schemes support a small set of homomorphic operations (e.g., addition, multiplication, rotation). Expressing complex functions often requires mathematical approximations.
- (3) **Performance tuning:** Achieving efficient execution requires careful management of noise growth, message precision and encryption parameters, which is non-trivial even for experts.



When it comes to PPML workloads, additional difficulties emerge:

- Neural networks involve many **non-linear activations**, making them hard to express efficiently in HE.
- Optimizing **data layout** for SIMD encoding is crucial for large tensor operations in the deeply nested computation graphs of neural networks, but doing so manually is extremely challenging.
- For CKKS—the most widely used FHE scheme for ML—proper **scale management** and **bootstrapping placement** are essential for both correctness and efficiency, and remain non-trivial even for experts.

5.1.2 Packing Optimization. Efficient packing is critical for leveraging the SIMD capabilities provided by RLWE-based schemes (e.g., BFV, BGV, CKKS). Several operator-level, expert-designed encoding strategies have been discussed in Section 3.1.2. Compiler-level packing primarily focuses on *SIMD encoding* (or vectorization), which offers more

flexible data movement semantics (i.e., Rotate operations). Over time, these compiler techniques have evolved from domain-specific heuristics to more general, automated strategies that jointly optimize vectorization and data layout.

Expert-driven and fixed-layout packing. Early systems such as CHET [50] automated the selection of encryption parameters and data layouts for tensor circuits using heuristics tailored to PPML workloads. CHET introduced the *CipherTensor* abstraction, which encapsulates a fixed set of supported layouts, each associated with a dedicated set of algorithmic kernels. EVA [49] extended this approach by introducing a domain-specific language (DSL) called Encrypted Vector Arithmetic (EVA). Nevertheless, both CHET and EVA still required developers to manually restructure applications into EVA’s batched execution model, limiting ease-of-use and efficiency.

Flexible layouts and automatic vectorization. To reduce reliance on manual layout design, later systems pursued more flexible and automated strategies. AHEC [32] extended the *Tile DSL* from PlaidML [173] to represent computations as element-wise operations suitable for vectorization, but delegated layout decisions entirely to the DSL, missing FHE-specific optimizations. Porcupine [47] adopted sketch-based synthesis to generate rotation-efficient kernels, though it requires user-provided sketches. Coyote [139] removed this barrier, enabling fully automatic vectorization for general programs by optimizing layout and rotations using a cost model to minimize latency and ciphertext noise growth. HECO [174] proposed a batching abstraction called *BatchedSecrets*, which avoids masking and supports operation chaining despite introducing some rotate-and-sum overhead.

Fixed row- or column-major layouts often lead to sub-optimal rotation patterns. To address this, recent compilers such as HELayer [3] and Fhelipe [110] introduce expressive layout abstractions that open up broader optimization opportunities for automatic packing. HELayer proposes a *tile tensor* design, which generalizes layout support using a unified set of algorithms. Fhelipe introduces a flexible tensor layout framework that supports arbitrary dimension orders, interleaving, and gaps. It explores a wider optimization space through an analytical layout selection algorithm that minimizes layout conversions and maximizes packing density. FlexHE [191] further enables generating and optimizing coefficient encoding-based 2PC-compatible HE kernels from einsum-like DNN representation.

5.1.3 Scale Management and Bootstrapping Placement. In CKKS [38], real-valued vectors are encoded into plaintext polynomials, where each value is scaled to an integer (the significand) with a fixed-point *scale*. RNS-CKKS [37] encrypts these polynomials under a ciphertext modulus composed of multiple RNS limbs, and the total *scale capacity* equals their product. Homomorphic multiplication inflates the scale, necessitating Rescale to keep it within capacity. In addition, ciphertexts must be at the same *level* (i.e., same number of limbs), requiring ModSwitch when levels mismatch. As computation proceeds, accumulated scale can exceed capacity, triggering the need for *bootstrapping* [36].

In general, effective *scale management* involves (1) selecting appropriate scaling factors and (2) determining the optimal insertion points for Rescale and ModSwitch operations across the computational graph to minimize overall latency. Optimal *bootstrapping placement* builds upon this scale management strategy, with the shared goal of latency reduction, particularly for DNN workloads. The development roadmap of PPML-oriented CKKS compilers that address these challenges is summarized in Figure 8.

Scale management-only compilers. CHET [50] provides only a profile-guided optimization for selecting scaling factors, without addressing the placement of Rescale and ModSwitch operations. EVA [49] introduces the first coarse-grained scale management strategy by defining a *waterline*, which is the minimum scale required to preserve message integrity. It performs Rescale operations reactively whenever the rescaled value remains above the waterline, passively reducing the ciphertext scale as much as possible. The insight behind EVA is that smaller ciphertext scales enable the use of smaller HE parameters, which in turn improves overall performance. HECATE [118] further advances scale

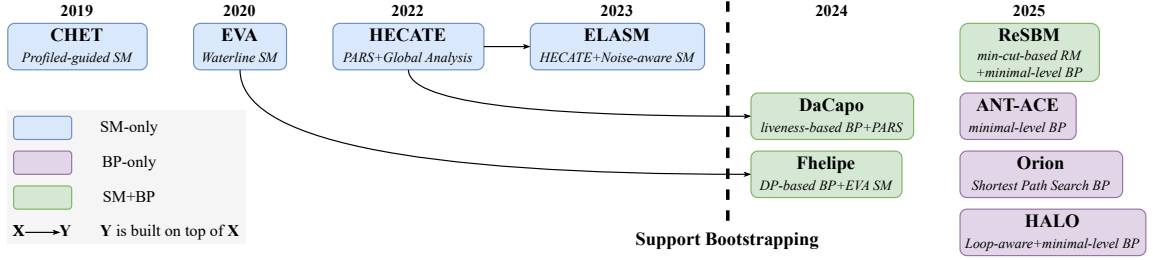


Fig. 8. Roadmap of CKKS compilers focusing on scale management (SM) and bootstrapping placement (BP) optimizations. For each node, the top half shows the **compiler name** in bold, and the bottom half summarizes its *key techniques* in italics.

management by recognizing the cascading impact of Rescale on the performance of subsequent operations. Since the computational complexity of homomorphic operations is directly affected by the ciphertext level, HECATE introduces a fine-grained, performance-aware scale management. It proactively inserts Rescale (referred to as downscale or PARS) to expand the optimization space for minimizing end-to-end latency. Unlike EVA’s two-phase approach, which first performs rescaling and then aligns levels, HECATE considers Rescale and ModSwitch operations holistically through hill-climbing-based space exploration. Building on HECATE, ELASM [117] introduces the first noise-aware scale management strategy. It balances latency and noise growth by configuring a more robust waterline to prevent scale underflow, which could otherwise affect the quality of service in PPML workloads.

Bootstrapping-capable compilers. The multiplicative depth required by DNNs often exceeds the limits of leveled HE, necessitating the most expensive FHE operation—bootstrapping—to enable more computation. Recent HE compilers have begun to support automatic bootstrapping placement to address this challenge. Contrary to bootstrapping increases the latency of subsequent operations. Therefore, effective bootstrapping management must be tightly integrated with scale management, especially given the NP-hard nature of optimal bootstrapping placement [147].

HECATE and ELASM determine optimal scale management plans by evaluating the cumulative cost of candidate plans using hill-climbing-based space exploration. However, this global optimization incurs high compilation overhead; for instance, compiling LeNet-5 takes over 300 seconds [117, 118], rendering the approach impractical for larger models. As a result, recent bootstrapping-capable compilers typically avoid global analysis for scale management.

Fhelipe [110] adopts a dynamic programming-based approach to generate bootstrapping plans based on EVA’s waterline-driven scale management. DaCapo [39] leverages liveness analysis to guide bootstrapping placement based on PARS [118] for scale management. However, both Fhelipe and DaCapo elevate ciphertexts to the maximum level without considering the latency implications of bootstrapping on subsequent operations, often resulting in sub-optimal plans. ReSBM [130] addresses this limitation by elevating ciphertexts only to the minimal required level, guided by its proposed min-cut-based scale management strategy. It is a region-based compiler that tightly coordinates scale management and bootstrapping placement for both compile-time and runtime efficiency. HALO [40] and ANT-ACE [122] also adopt this minimal-level bootstrapping strategy but do not explicitly elaborate on their scale management approaches. Notably, HALO is the first to propose loop-aware bootstrapping placement, enabling optimized code generation within non-unrolled loops. Orion [59] allows more flexible bootstrapping initiation points within the circuit. It formulates the placement problem as a shortest-path search over a DAG and incorporates network-specific heuristics to guide the search. However, Orion does not integrate scale management with bootstrapping; instead, it relies on a conservative, manually configured scale management [18, 101].



Takeaways of PPML-oriented HE compilers:

- Manually developing performant FHE programs is error-prone and labor-intensive due to limited operations and complex trade-offs among correctness, latency, and noise. These challenges are further exacerbated in PPML workloads, motivating PPML-specific HE compilers.
- Compiler-driven packing optimizations have progressed from fixed-layout heuristics to automatic, layout-flexible strategies that substantially improve utilization for SIMD encoding.
- Scale management and bootstrapping placement are tightly coupled challenges. Recent compilers adopt level-aware cost models and region-based scheduling to jointly minimize scale consumption and latency.
- Despite these advances, existing approaches still suffer from scalability limitations, such as long compilation times and poor support for complex model architectures like Transformers.

5.2 GPU Optimization

One of the primary obstacles to the widespread adoption of HE is the significant performance gap between homomorphic and plaintext computations. For example, even the most advanced implementations of CNN inference on the CIFAR-10 dataset still require hundreds to thousands of seconds per inference on CPU [14, 102, 113, 162]. In hybrid schemes that combine HE with MPC, a common strategy to reduce communication overhead is to offload as much linear computation as possible to the HE domain. However, this often increases the computational cost of leveled HE operations, making GPU acceleration essential. Given that GPUs are the dominant compute resource in modern cloud infrastructures, accelerating PPML on GPUs is a natural and promising direction. This section discusses GPU optimization from two perspectives: operation-level acceleration and PPML system-level optimization. We conclude by summarizing the execution time of representative PPML workloads reported in existing GPU-accelerated implementations.

5.2.1 Operation-Level Optimization. Early efforts in GPU acceleration for HE primarily focused on polynomial arithmetic, with particular attention to the Number Theoretic Transform (NTT), one of the most computationally intensive operators in HE. Recent works continue to target NTT for optimization. HE-Booster [179] achieves fine-grained NTT by leveraging inter-thread local synchronization to increase overlap and reduce barrier stalls. Fan et al. [62] explores how to efficiently utilize CUDA cores by implementing NTT using FP64 arithmetic units. TensorFHE [64] utilizes Tensor Core Units (TCUs) with INT8 components to accelerate NTT computation. WarpDrive [61] reduces warp-level stalls by concurrently scheduling TCUs and CUDA cores. Neo [95] further exploits TCUs by transforming scalar and element-wise multiplications into matrix multiplications using the FP64 components in TCUs.

Key switching. Key switching (KeySwitch) is a maintenance operation in HE schemes that transforms intermediate ciphertexts, whose associated decryption keys have changed due to prior homomorphic operations, back into a form decryptable by the user’s secret key. In practice, KeySwitch often becomes the performance bottleneck in HE workloads.

Prior work has accelerated KeySwitch by optimizing the underlying polynomial arithmetic. For example, Over100x [97] adopts an improved hybrid key switching (HKS) algorithm proposed in [80]. Phantom [166, 189] and WarpDrive enhance HKS further by decomposing the input ciphertexts on demand. Neo explores GPU acceleration for the state-of-the-art KLSS-based KeySwitch algorithm [106].

Bootstrapping. As discussed in Sections 3.3 and 5.1.3, bootstrapping is another essential maintenance operation in HE schemes, particularly for deep workloads such as DNNs. As the most time-consuming component, bootstrapping is commonly used as a benchmark to evaluate the effectiveness in recent studies.

Over100x [97] presents the first GPU-accelerated CKKS bootstrapping implementation. Based on the observation that element-wise operations dominate the bootstrapping cost, they propose memory-centric optimizations such as kernel fusion and reordering. TensorFHE introduces packed bootstrapping to improve throughput. Cheddar [103] implements an advanced CKKS bootstrapping algorithm [19] on GPU. Other GPU-accelerated CKKS efforts have not directly optimized bootstrapping itself, instead focusing on accelerating lower-level primitives as discussed above.

Early GPU acceleration efforts for FHEW/TFHE focused on gate bootstrapping. Zama’s TFHE-rs advances this by supporting more practical LUT-based PBS (see Section 4.2.3). Given the relatively small parameters in TFHE (with polynomial lengths not exceeding 1024), TFHE-rs adopts a single-thread-block-per-SM strategy to execute the entire PBS. Xiao et al. [182] further design a multi-thread-block PBS kernel optimized for small batch sizes. VeloFHE [167] introduces a hybrid four-step NTT algorithm and applies multi-level memory-centric optimizations.

5.2.2 System-Level Optimization. At the system level, several studies have explored hardware extensions, scheduling strategies, and heterogeneous optimizations to enhance HE performance on GPUs. GME [168] proposes a custom hardware design based on the AMD GPU architecture to accelerate polynomial operations. BoostCom [192] improves GPU utilization by using CPU multithreading to dispatch GPU kernels for BGV scheme comparison operations. CAT [123] proposes a task scheduling strategy that combines kernel segmentation with the CUDA multi-stream technique. Anaheim [104] combines GPU with processing-in-memory (PIM) technology to form a heterogeneous accelerator, offloading memory-bound computations to the PIM side for significant performance gains in real workloads.

5.2.3 PPML Workload. Commonly used workloads for evaluating GPU-accelerated HE include:

- **HELR:** Homomorphic logistic regression. *HELR-train* refers to the training workload, which executes 32 iterations on 1024 batches of 14×14 grayscale images [79]. Performance is typically reported in the execution time per iteration. *HELR-infer* refers to the inference workload, though its exact definition varies across studies.
- **RNN [155]:** Recurrent neural network inference. A single ciphertext encodes 32 embeddings, each of length 128. The benchmark runs 200 iterations of an RNN layer with 128 units.
- **HECNN:** HE-friendly convolution neural network inference. *CryptoNets* [72] refers to the 5-layer CNN which replaces non-linear activations with squaring functions. Similar lightweight CNNs have also been retrained for TFHE-based inference [22, 83].
- **ResNet:** Convolution neural network inference using ResNet architecture. *ResNet-20/32/56* refers to the inference of a single CIFAR-10 image [113]. *ResNet-18* refers to the inference of a single ImageNet image.

A summary of recent GPU-accelerated HE frameworks is provided in Table 7, including the accessibility of their source code. Performance results for PPML benchmarks are summarized in Figure 9. Currently, only NEXUS [197] and de Castro et al. [52] report GPU-accelerated LLM inference using pure HE. In particular, de Castro et al. [52] evaluates one decoding step of GPT-2 at token position 128, which takes approximately 105 seconds on an A100-80GB GPU. This result is later optimized to 69 seconds [51].

Table 7. Overview of GPU-accelerated HE frameworks.^a

Literature	Year	Scheme				Support boot.	PPML benchmark ^b	Accessibility	
		BFV	BGV	CKKS	TFHE/FHEW			Code	Exe.
Over100x [97]	2021	○	○	●	○	●	HELR-train	●	●
HE-Booster [179]	2023	○	●	●	○	○	CryptoNets	○	○
TensorFHE [64]	2023	○	○	●	○	●	HELR-train, RNN, ResNet-20	○	●
GME [168]	2023	○	○	●	○	●	HELR-train, ResNet-20	○	○
BoostCom [192]	2023	○	●	○	○	○	MLP	○	○
Phantom [166, 189]	2024	●	●	●	○	○	-	●	●
Cheddar [103]	2024	○	○	●	○	●	HELR-train, RNN, ResNet-20	●	●
de Castro et al. [51, 52]	2024	○	○	●	○	●	GPT-2	●	●
ArctyrEX	2025	○	○	○	●	●	HELR-infer, HECNN [22], AES-128-Dec [†]	○	○
Xiao et al. [182]	2025	○	○	○	●	●	HECNN [83]	○	○
CAT [123]	2025	●	●	●	○	○	PDQ [†]	●	●
HEonGPU [146]	2025	●	○	●	●	●	PIR [†]	●	●
WarpDrive [61]	2025	○	○	●	○	●	HELR-train, ResNet-20, AES-128-Dec [†]	○	○
Anaheim [104]	2025	○	○	●	○	●	HELR-train, RNN, ResNet-20, ResNet-18	○	○
Neo [95]	2025	○	○	●	○	●	HELR-train, ResNet-20/32/56	○	○

^a "●" = full, "◐" = partial/limited, "○" = none. For "Scheme" columns, this refers to support; for "Accessibility" columns, it indicates availability (open-source or binary).

^b "PPML benchmark" refers to those defined in Section 5.2.3 or explicitly named in the original papers.

[†] Not a PPML task: PIR = Private Information Retrieval, PDQ = Private Dataset Query, AES-128-Dec = AES-128 Decryption/Transciphering.

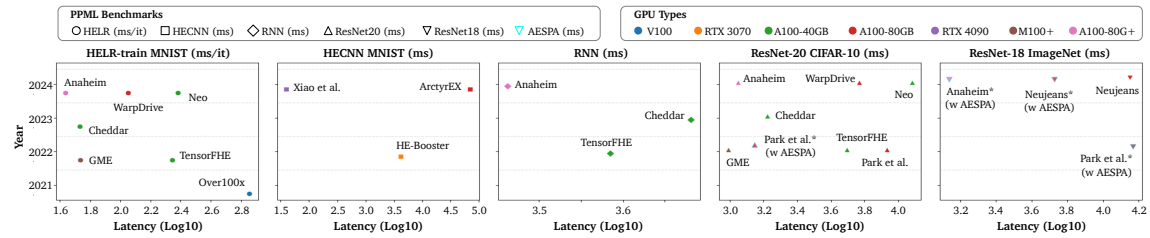


Fig. 9. Execution time of GPU-accelerated PPML workloads (as defined in Section 5.2.3). Note that Neujeans [96] and Park et al. [150] report only the latency after GPU acceleration. Methods based on AESPA are highlighted. M100+ and A100-80GB+ indicate systems with custom extensions to the original GPUs.

5.3 Implementation and Library

We summarize the open-source PPML libraries in Table 8. These libraries use a range of cryptographic techniques, including SS, HE, and OT, to ensure that computations are performed securely and without revealing sensitive data.

6 Discussions

This literature review highlights that while substantial progress has been made, current PPML frameworks still encounter efficiency bottlenecks, underscoring the need for further efforts in this field. Here, we put forward several discussion points regarding the future development of PPML, which we hope may serve as a starting point for broader research.

6.1 What are the Challenges and Opportunities of Cross-level Optimization?

For the future optimization direction, we argue that full-stack, i.e., cross-level optimization holds great promise. There arises a question: "Can different levels be decoupled from each other? What are the challenges and unique opportunities when considering cross-level optimizations?"

Table 8. Open-source PPML implementations and libraries.

Library	Time	Provided Protocols	Front-end Language	Back-end Language	Repository Link
HElib [78]	2013	HE (BGV, CKKS)	C++	C++	https://github.com/homenc/HElib
python-paillier [48]	2014	HE (Paillier)	Python	Python	https://github.com/data61/python-paillier
Pyfhel [88]	2017	HE (BGV, BFV, CKKS)	Python	C++	https://github.com/ibarrond/Pyfhel
SEAL [163]	2018	HE (BGV, BFV, CKKS)	C++	C++	https://github.com/microsoft/SEAL
SEAL-Python [163]	2019	HE (BGV, BFV, CKKS)	Python	C++	https://github.com/Huelse/SEAL-Python
OpenFHE [6]	2021	HE (BGV, BFV, CKKS)	C++, Python	C++	https://github.com/openfheorg/openfhe-development
Lattigo [1]	2024	HE (BGV, BFV, CKKS)	Go	Go	https://github.com/tuneinsight/lattigo
TFHE-rs [4]	2024	HE (TFHE)	C, Rust	Rust	https://github.com/zama-ai/tfhe-rs
ABY [53]	2015	MPC	C++	C++	https://github.com/encryptogroup/ABY
EMP-toolkit [177]	2016	MPC	C++	C++	https://github.com/emp-toolkit
ABY3 [141]	2018	MPC	C++	C++	https://github.com/ladnir/aby3
CrypTen [107]	2019	MPC	Python	Python	https://github.com/facebookresearch/CrypTen
MOTION [25]	2022	MPC	C++	C++	https://github.com/encryptogroup/MOTION
MP-SPDZ [99]	2016	MPC, HE	Python	C++	https://github.com/data61/MP-SPDZ
EzPC [29]	2019	MPC, HE	C++	C++	https://github.com/mpc-msri/EzPC
Secretflow [138]	2022	MPC, HE	Python	C++	https://github.com/secretflow/secretflow

Discussion point 1: Protocol-model co-optimization. Model-level optimization like quantization usually requires the support of specifically designed protocols. Protocol-model co-optimization is essential to achieving maximal benefits. We provide the following cases: **1) Quantization cannot directly help PPML.** Although quantization drastically reduces the computation and memory overhead, it can introduce extra expensive online costs, including bit width extension, truncation, re-quantization, etc. Existing studies [183, 187, 196] propose PPML-friendly quantization algorithms and protocol optimizations like protocol fusion to minimize the overall costs. **2) Non-linear layer optimization cannot help reduce total communication.** Numerous studies focus on ReLU/GeLU pruning for lower online costs; however, they all ignore the importance of reducing total costs for MLaaS, including online and pre-processing costs. CoPriv [195] integrates Winograd convolution protocol, ReLU pruning, and layer fusion to cut down the costs of both linear and non-linear layers. We believe that in the era of MLaaS, overall efficiency matters most for practical deployment. **3) Effective exploitation of sparsity is essential for PPML efficiency gain.** Although the attention mechanism exhibits high sparsity in LLMs, direct pruning either leads to accuracy degradation or introduces additional overhead. Recently, MPCache [193] designs a static-dynamic pruning algorithm that carefully considers protocol costs, achieving efficient and accurate pruning. Other AI techniques such as neural architecture search (NAS) [140, 186, 194, 195], knowledge distillation (KD) [31, 54, 120, 194], and low-rank decomposition can be applied or optimized for PPML. Nevertheless, we argue that **extra training is not practical and scalable for LLMs. Therefore, post-training quantization, sparsity utilization, training-free decomposition, etc, should be prioritized.**

Discussion point 2: Protocol-system co-optimization. System-level optimization for PPML significantly impacts efficiency, yet it often lacks deep consideration of underlying protocols. This leads to sub-optimal performance, highlighting the need for protocol-system co-optimization, especially for **HE-based protocols**. We highlight the following: **1) Compiler-protocol co-design is key for efficient HE.** Manual FHE programming is challenging due to complex trade-offs in correctness, noise, packing, and latency, underscoring the need for compilers. While compilers improve packing, their efficacy often relies on expertly optimized protocol-level encoding [59, 110]. Conversely, current compiler-driven packing mostly uses SIMD encoding, neglecting potentially more efficient coefficient encoding. Bridging this gap by integrating protocol insights into compiler frameworks is a major opportunity. **2) ML-orient GPU components cannot directly accelerate PPML.** Modern GPUs with TCUs accelerate unencrypted ML, but their limited precision support poses challenges for the high-precision modular arithmetic required by HE [61, 64].

This demands co-design between cryptographic protocols and hardware capabilities. Future protocols may need to be redesigned to leverage TCU features, possibly through novel encoding or approximate operations. Synergistic efforts with model quantization are crucial to fully exploit GPU’s new components for encrypted computations.

6.2 Is There a Way to Compute Non-Linear Layers Directly Using HE without Polynomial Approximation?

OT-based non-linear layer brings significant communication overhead. In contrast, HE-based methods are more communication-efficient. In existing methods, polynomial approximation is necessary for non-linear layer computation. As mentioned, low-degree polynomials require extra training efforts, while high-degree polynomials lead to much higher online inference overhead. Hence, there is a natural question: “Can we directly compute non-linear layers using HE without polynomial approximation?” This question is still underexplored.

6.3 Discussions about PPML in the Era of LLMs: Unique Challenges and Our Views

Finally, we want to discuss PPML in the era of LLMs, which is much more challenging than prior private inference on CNNs and ViTs. For example, for an input with only 4 tokens, PUMA [56] even requires 122 s latency and 0.9 GB communication to generate 1 token, which is impractical for real-world applications. The key challenges of private LLM inference can be roughly summarized as follows: **1) Large-scale linear layers.** Unlike the matrix-vector multiplications commonly used in CNNs, LLMs involve lots of high-dimensional matrix multiplications in the embedding table, Attention layers, and FFN layers. For instance, in the GPT-2 base model, the up projection in the FFN layer requires a matrix multiplication of dimensions (*#tokens*, 768, 3072). **2) Complicated non-linear layers.** Different from simple ReLU in CNNs, LLMs consist of more complex non-linear functions like Softmax, GeLU, and SiLU, which require expensive exponential, tanh, and division protocols. Due to the large scale of the Transformer, GPT-2 inference requires about 3.9×10^6 point-wise GeLUs. **3) Complicate PPML-aware optimizations.** LLM optimization is much more complex than CNN and ViT, and training-free methods should be prioritized. As mentioned in MPCache [193], naive KV cache compression fails to deliver the desired efficiency improvements, and thus, PPML-friendly algorithms should be carefully designed. Furthermore, we also think parameter-efficient fine-tuning (PEFT), like LoRA for PPML-friendly LLM architecture design, can be a promising research direction. Moreover, due to the massive scale of LLMs, more severe challenges also arise at the system and protocol levels.

7 Conclusion

This survey provides a systematic literature review of PPML with structured and meticulous taxonomy, i.e., protocol-level, model-level, and system-level optimization. This survey also provides qualitative and quantitative comparisons of existing works with technical insights. Moreover, this survey discusses future research directions and highlights the necessity of integrating optimizations across protocol, model, and system levels, serving as a road map and white book for researchers in the PPML community.

References

- [1] 2024. Lattigo v6. Online: <https://github.com/tuneinsight/lattigo>. EPFL-LDS, Tune Insight SA.
- [2] Nitin Agrawal, Ali Shahin Shamsabadi, Matt J Kusner, and Adrià Gascón. 2019. QUOTIENT: Two-party secure neural network training and prediction. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 1231–1247.
- [3] Ehud Aharoni, Allon Adir, Moran Baruch, Nir Drucker, Gilad Ezov, Ariel Farkash, Lev Greenberg, Ramy Masalha, Guy Moshkovich, Dov Murik, et al. 2020. Helayers: A tile tensors framework for large neural networks on encrypted data. *arXiv preprint arXiv:2011.01805* (2020).

- [4] Zama AI. 2024. TFHE-rs: A pure rust implementation of the TFHE scheme for boolean and integer arithmetics over encrypted data. *Software available at <https://github.com/zama-ai/tfhe-rs>* (2024).
- [5] Yoshimasa Akimoto, Kazuto Fukuchi, Youhei Akimoto, and Jun Sakuma. 2023. Privformer: Privacy-preserving transformer with mpc. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 392–410.
- [6] Ahmad Al Badawi, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Ian Quah, Yuriy Polyakov, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Sponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. 2022. OpenFHE: Open-Source Fully Homomorphic Encryption Library. In *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography* (Los Angeles, CA, USA) (WAHC'22). Association for Computing Machinery, New York, NY, USA, 53–63. doi:10.1145/3560827.3563379
- [7] Amjad Almusaed, Ibrahim Yitmen, and Asaad Almssad. 2023. Enhancing smart home design with AI models: A case study of living spaces implementation review. *Energies* 16, 6 (2023), 2636.
- [8] Anthropic. 2025. Claude 4. <https://www.anthropic.com/news/claude-4>.
- [9] Wei Ao and Vishnu Naresh Boddeti. 2024. {AutoFHE}: Automated Adaption of {CNNs} for Efficient Evaluation over {FHE}. In *33rd USENIX Security Symposium (USENIX Security 24)*. 2173–2190.
- [10] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, and Damien Stehlé. 2024. Bootstrapping Bits with CKKS. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 94–123.
- [11] Youngjin Bae, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. 2025. Bootstrapping small integers with CKKS. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 330–360.
- [12] Donald Beaver. 1995. Precomputing oblivious transfer. In *Annual International Cryptology Conference*. Springer, 97–109.
- [13] Mihir Bellare, Viet Tung Hoang, Sriram Keelveedhi, and Phillip Rogaway. 2013. Efficient garbling from a fixed-key blockcipher. In *2013 IEEE Symposium on Security and Privacy*. IEEE, 478–492.
- [14] Adrien Benamira, Tristan Guérard, Thomas Peyrin, and Sayandeep Saha. 2023. TT-TFHE: a torus fully homomorphic encryption-friendly neural network architecture. *arXiv preprint arXiv:2302.01584* (2023).
- [15] Yuchen Bian, Jiaji Huang, Xingyu Cai, Jiahong Yuan, and Kenneth Church. 2021. On attention redundancy: A comprehensive study. In *Proceedings of the 2021 conference of the north american chapter of the association for computational linguistics: human language technologies*. 930–945.
- [16] Fabian Boemer, Anamaria Costache, Rosario Cammarota, and Casimir Wierzynski. 2019. nGraph-HE2: A high-throughput framework for neural network inference on encrypted data. In *Proceedings of the 7th ACM workshop on encrypted computing & applied homomorphic cryptography*. 45–56.
- [17] Fabian Boemer, Yixing Lao, Rosario Cammarota, and Casimir Wierzynski. 2019. nGraph-HE: a graph compiler for deep learning on homomorphically encrypted data. In *Proceedings of the 16th ACM international conference on computing frontiers*. 3–13.
- [18] Jean-Philippe Bossuat, Christian Mouchet, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2021. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 587–617.
- [19] Jean-Philippe Bossuat, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2022. Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In *International Conference on Applied Cryptography and Network Security*. Springer, 521–541.
- [20] Christina Boura, Nicolas Gama, and Mariya Georgieva. 2018. Chimera: a unified framework for B/FV, TFHE and HEAAN fully homomorphic encryption and predictions for deep learning. *IACR Cryptol. ePrint Arch.* 2018 (2018), 758.
- [21] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. 2019. Simulating homomorphic evaluation of deep learning predictions. In *International Symposium on Cyber Security Cryptography and Machine Learning*. Springer, 212–230.
- [22] Florian Bourse, Michele Minelli, Matthias Minihold, and Pascal Paillier. 2018. Fast homomorphic evaluation of deep discretized neural networks. In *Advances in Cryptology—CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part III* 38. Springer, 483–512.
- [23] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. 2018. Compressing vector OLE. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 896–912.
- [24] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2014. (Leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)* 6, 3 (2014), 1–36.
- [25] Lennart Braun, Daniel Demmler, Thomas Schneider, and Oleksandr Tkachenko. 2020. MOTION - A Framework for Mixed-Protocol Multi-Party Computation. *Cryptology ePrint Archive*, Paper 2020/1137. doi:10.1145/3490390
- [26] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [27] Megha Balyi, Harsh Chaudhari, Arpita Patra, and Ajith Suresh. 2019. FLASH: Fast and robust framework for privacy-preserving machine learning. *Cryptology ePrint Archive* (2019).
- [28] Yifei Cai, Qiao Zhang, Rui Ning, Chunsheng Xin, and Hongyi Wu. 2022. Hunter: HE-Friendly Structured Pruning for Efficient Privacy-Preserving Deep Learning (ASIA CCS '22). Association for Computing Machinery, New York, NY, USA, 931–945. doi:10.1145/3488932.3517401
- [29] Nishanth Chandran, Divya Gupta, Aseem Rastogi, Rahul Sharma, and Shardul Tripathi. 2019. EzPC: Programmable and efficient secure two-party computation for machine learning. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 496–511.

- [30] Harsh Chaudhari, Rahul Rachuri, and Ajith Suresh. 2019. Trident: Efficient 4pc framework for privacy preserving machine learning. *arXiv preprint arXiv:1912.02631* (2019).
- [31] Dake Chen, Yuke Zhang, Souvik Kundu, Chenghao Li, and Peter A Beerel. 2023. RNA-ViT: Reduced-Dimension Approximate Normalized Attention Vision Transformers for Latency Efficient Private Inference. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [32] Huili Chen, Rosario Cammarota, Felipe Valencia, Francesco Regazzoni, and Farinaz Koushanfar. 2020. Ahec: End-to-end compiler framework for privacy-preserving machine learning acceleration. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [33] Qiguang Chen, Libo Qin, Jinhao Liu, Dengyun Peng, Jiannan Guan, Peng Wang, Mengkang Hu, Yuhang Zhou, Te Gao, and Wanxiang Che. 2025. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models. *arXiv preprint arXiv:2503.09567* (2025).
- [34] Tianyu Chen, Hangbo Bao, Shaohan Huang, Li Dong, Binxing Jiao, Daxin Jiang, Haoyi Zhou, Jianxin Li, and Furu Wei. 2022. The-x: Privacy-preserving transformer inference with homomorphic encryption. *arXiv preprint arXiv:2206.00216* (2022).
- [35] Ke Cheng, Ning Xi, Ximeng Liu, Xinghui Zhu, Haichang Gao, Zhiwei Zhang, and Yulong Shen. 2023. Private Inference for Deep Neural Networks: A Secure, Adaptive, and Efficient Realization. *IEEE Trans. Comput.* 72, 12 (2023), 3519–3531. doi:10.1109/TC.2023.3305754
- [36] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. Bootstrapping for approximate homomorphic encryption. In *Advances in Cryptology—EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29–May 3, 2018 Proceedings, Part I 37*. Springer, 360–384.
- [37] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. A full RNS variant of approximate homomorphic encryption. In *International Conference on Selected Areas in Cryptography*. Springer, 347–368.
- [38] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology—ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3–7, 2017, Proceedings, Part I 23*. Springer, 409–437.
- [39] Seonyoung Cheon, Yongwoo Lee, Dongkwan Kim, Ju Min Lee, Sunchul Jung, Taekyung Kim, Dongyoon Lee, and Hanjun Kim. 2024. {DaCapo}: Automatic Bootstrapping Management for Efficient Fully Homomorphic Encryption. In *33rd USENIX Security Symposium (USENIX Security 24)*. 6993–7010.
- [40] Seonyoung Cheon, Yongwoo Lee, Hoyun Youm, Dongkwan Kim, Sungwoo Yun, Kunmo Jeong, Dongyoon Lee, and Hanjun Kim. 2025. HALO: Loop-aware Bootstrapping Management for Fully Homomorphic Encryption. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 572–585.
- [41] Diego Chialva and Ann Doms. 2018. Conditionals in homomorphic encryption and machine learning applications. *arXiv preprint arXiv:1810.12380* (2018).
- [42] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2020. TFHE: fast fully homomorphic encryption over the torus. *Journal of Cryptology* 33, 1 (2020), 34–91.
- [43] Ilaria Chillotti, Marc Joye, and Pascal Paillier. 2021. Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In *Cyber Security Cryptography and Machine Learning: 5th International Symposium, CSCML 2021, Be'er Sheva, Israel, July 8–9, 2021, Proceedings 5*. Springer, 1–19.
- [44] Minsu Cho, Zahra Ghodsi, Brandon Reagen, Siddharth Garg, and Chinmay Hegde. 2022. Sphynx: A Deep Neural Network Design for Private Inference. *IEEE Security & Privacy* 20, 5 (Sept. 2022), 22–34. doi:10.1109/msec.2022.3165475
- [45] Minsu Cho, Ameya Joshi, Brandon Reagen, Siddharth Garg, and Chinmay Hegde. 2022. Selective network linearization for efficient private inference. In *International Conference on Machine Learning*. PMLR, 3947–3961.
- [46] Geoffroy Couteau and Clément Ducros. 2023. Pseudorandom correlation functions from variable-density LPN, revisited. In *IACR International Conference on Public-Key Cryptography*. Springer, 221–250.
- [47] Meghan Cowan, Deeksha Dangwal, Armin Alaghi, Caroline Trippel, Vincent T Lee, and Brandon Reagen. 2021. Porcupine: A synthesizing compiler for vectorized homomorphic encryption. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 375–389.
- [48] CSIRO's Data61. 2013. Python Paillier Library. <https://github.com/data61/python-paillier>.
- [49] Roshan Dathathri, Blagovesta Kostova, Olli Saarikivi, Wei Dai, Kim Laine, and Madan Musuvathi. 2020. EVA: An encrypted vector arithmetic language and compiler for efficient homomorphic computation. In *Proceedings of the 41st ACM SIGPLAN conference on programming language design and implementation*. 546–561.
- [50] Roshan Dathathri, Olli Saarikivi, Hao Chen, Kim Laine, Kristin Lauter, Saeed Maleki, Madanlal Musuvathi, and Todd Mytkowicz. 2019. CHET: an optimizing compiler for fully-homomorphic neural-network inferencing. In *Proceedings of the 40th ACM SIGPLAN conference on programming language design and implementation*. 142–156.
- [51] Leo de Castro, Daniel Escudero, Adya Agrawal, Antigoni Polychroniadou, and Manuela Veloso. [n. d.]. EncryptedLLM: Privacy-Preserving Large Language Model Inference via GPU-Accelerated Fully Homomorphic Encryption. In *Forty-second International Conference on Machine Learning*.
- [52] Leo de Castro, Antigoni Polychroniadou, and Daniel Escudero. 2024. Privacy-Preserving Large Language Model Inference via GPU-Accelerated Fully Homomorphic Encryption. In *Neurips Safe Generative AI Workshop 2024*.
- [53] Daniel Demmler, Thomas Schneider, and Michael Zohner. 2015. ABY-A framework for efficient mixed-protocol secure two-party computation.. In *NDSS*.

- [54] Naren Dhyani, Jianqiao Mo, Minsu Cho, Ameya Joshi, Siddharth Garg, Brandon Reagen, and Chinmay Hegde. 2023. PriViT: Vision Transformers for Fast Private Inference. *arXiv preprint arXiv:2310.04604* (2023).
- [55] Abdulrahman Diaa, Lucas Fenaux, Thomas Humphries, Marian Dietz, Faezeh Ebrahimiaghazani, Bailey Kacsmar, Xinda Li, Nils Lukas, Rasoul Akhavan Mahdavi, Simon Oya, et al. 2024. Fast and private inference of deep neural networks by co-designing activation functions. In *33rd USENIX Security Symposium (USENIX Security 24)*. 2191–2208.
- [56] Ye Dong, Wen-jie Lu, Yancheng Zheng, Haoqi Wu, Derun Zhao, Jin Tan, Zhicong Huang, Cheng Hong, Tao Wei, and Wenguang Chen. 2023. Puma: Secure inference of llama-7b in five minutes. *arXiv preprint arXiv:2307.12533* (2023).
- [57] Zhen Dong, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. 2019. Hawq: Hessian aware quantization of neural networks with mixed-precision. In *Proceedings of the IEEE/CVF international conference on computer vision*. 293–302.
- [58] Alexey Dosovitskiy. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [59] Austin Ebel, Karthik Garimella, and Brandon Reagen. 2023. Orion: A Fully Homomorphic Encryption Compiler for Private Deep Neural Network Inference. *arXiv preprint arXiv:2311.03470* (2023).
- [60] Fabian Boemer et al. 2020. MP2ML: A mixed-protocol machine learning framework for private inference. In *Proceedings of the 15th International Conference on Availability, Reliability and Security*. 1–10.
- [61] Guang Fan, Mingzhe Zhang, Fangyu Zheng, Shengyu Fan, Tian Zhou, Xianglong Deng, Wenxu Tang, Liang Kong, Yixuan Song, and Shoumeng Yan. 2025. WarpDrive: GPU-Based Fully Homomorphic Encryption Acceleration Leveraging Tensor and CUDA Cores. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1187–1200.
- [62] Guang Fan, Fangyu Zheng, Lipeng Wan, Lili Gao, Yuan Zhao, Jiankuo Dong, Yixuan Song, Yuewu Wang, and Jingqiang Lin. 2023. Towards faster fully homomorphic encryption implementation with integer and floating-point computing power of GPUs. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 798–808.
- [63] Junfeng Fan and Frederik Vercauteren. 2012. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive* (2012).
- [64] Shengyu Fan, Zhiwei Wang, Weizhi Xu, Rui Hou, Dan Meng, and Mingzhe Zhang. 2023. Tensorfhe: Achieving practical computation on encrypted data using gpgpu. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 922–934.
- [65] Lars Folkerts, Charles Gouert, and Nektarios Georgios Tsoutsos. 2021. REDsec: Running encrypted discretized neural networks in seconds. *Cryptology ePrint Archive* (2021).
- [66] Jordan Frery, Andrei Stoian, Roman Bredehoft, Luis Montero, Celia Kherfallah, Benoit Chevallier-Mames, and Arthur Meyre. 2023. Privacy-preserving tree-based inference with fully homomorphic encryption. *Cryptology ePrint Archive* (2023).
- [67] Vinod Ganesan, Anwesh Bhattacharya, Pratyush Kumar, Divya Gupta, Rahul Sharma, and Nishanth Chandran. 2022. Efficient ml models for practical secure inference. *arXiv preprint arXiv:2209.00411* (2022).
- [68] Xian Gao, Peixiong He, Yi Zhou, and Xiao Qin. 2024. Artificial Intelligence Applications in Smart Healthcare: A Survey. *Future Internet* 16, 9 (2024), 308.
- [69] Juan Garay, Berry Schoenmakers, and José Villegas. 2007. Practical and secure solutions for integer comparison. In *Public Key Cryptography–PKC 2007: 10th International Conference on Practice and Theory in Public-Key Cryptography Beijing, China, April 16–20, 2007. Proceedings 10*. Springer, 330–342.
- [70] Zahra Ghodsi, Nandan Kumar Jha, Brandon Reagen, and Siddharth Garg. 2021. Circa: Stochastic relus for private deep learning. *Advances in Neural Information Processing Systems* 34 (2021), 2241–2252.
- [71] Zahra Ghodsi, Akshaj Kumar Veldanda, Brandon Reagen, and Siddharth Garg. 2020. Cryptonas: Private inference on a relu budget. *Advances in Neural Information Processing Systems* 33 (2020), 16961–16971.
- [72] Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing. 2016. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In *International conference on machine learning*. PMLR, 201–210.
- [73] Yakir Gorski, Amir Jevnisek, and Shai Avidan. 2023. Securing Neural Networks with Knapsack Optimization. *arXiv preprint arXiv:2304.10442* (2023).
- [74] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [75] Meng-Hao Guo, Zheng-Ning Liu, Tai-Jiang Mu, and Shi-Min Hu. 2022. Beyond self-attention: External attention using two linear layers for visual tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 5 (2022), 5436–5447.
- [76] Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. 2023. Sigma: Secure gpt inference with function secret sharing. *Cryptology ePrint Archive* (2023).
- [77] Kanav Gupta, Deepak Kumaraswamy, Nishanth Chandran, and Divya Gupta. 2022. Llama: A low latency math library for secure inference. *Cryptology ePrint Archive* (2022).
- [78] Shai Halevi and Victor Shoup. 2020. Design and implementation of HELib: a homomorphic encryption library. *Cryptology ePrint Archive*, Paper 2020/1481. <https://eprint.iacr.org/2020/1481>
- [79] KyooHyung Han, Seungwan Hong, Jung Hee Cheon, and Daejun Park. 2019. Logistic regression on homomorphic encrypted data at scale. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 9466–9471.
- [80] KyooHyung Han and Dohyeon Ki. 2020. Better bootstrapping for approximate homomorphic encryption. In *Cryptographers’ Track at the RSA Conference*. Springer, 364–390.

- [81] Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. 2022. Iron: Private inference on transformers. *Advances in neural information processing systems* 35 (2022), 15718–15731.
- [82] Jiaxing He, Kang Yang, Guofeng Tang, Zhangjie Huang, Li Lin, Changzheng Wei, Ying Yan, and Wei Wang. 2024. Rhombus: Fast Homomorphic Matrix-Vector Multiplication for Secure Two-Party Inference. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2490–2504.
- [83] Ming-Chien Ho, Yu-Te Ku, Yu Xiao, Feng-Hao Liu, Chih-Fan Hsu, Ming-Ching Chang, Shih-Hao Hung, and Wei-Chao Chen. 2024. Efficient Design of FHEW/TFHE Bootstrapping Implementation with Scalable Parameters. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [84] Xiaoyang Hou, Jian Liu, Jingyu Li, Yuhua Li, Wen-jie Lu, Cheng Hong, and Kui Ren. 2023. Ciphertgpt: Secure two-party gpt inference. *Cryptology ePrint Archive* (2023).
- [85] Jie Huang and Kevin Chen-Chuan Chang. 2022. Towards reasoning in large language models: A survey. *arXiv preprint arXiv:2212.10403* (2022).
- [86] Zhicong Huang, Wen-jie Lu, Cheng Hong, and Jiansheng Ding. 2022. Cheetah: Lean and fast secure {Two-Party} deep neural network inference. In *31st USENIX Security Symposium (USENIX Security 22)*. 809–826.
- [87] Siam Umar Hussain, Mojan Javaheripi, Mohammad Samragh, and Farinaz Koushanfar. 2021. Coinn: Crypto/ml codesign for oblivious inference via neural networks. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 3266–3281.
- [88] Alberto Ibarro and Alexander Viand. 2021. Pyfhel: Python for homomorphic encryption libraries. In *Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. 11–16.
- [89] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. 2003. Extending oblivious transfers efficiently. In *Annual International Cryptology Conference*. Springer, 145–161.
- [90] Neha Jawalkar, Kanav Gupta, Arkaprava Basu, Nishanth Chandran, Divya Gupta, and Rahul Sharma. 2024. Orca: Fss-based secure training and inference with gpus. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 597–616.
- [91] Nandan Kumar Jha, Zahra Ghodsi, Siddharth Garg, and Brandon Reagen. 2021. Deepreduce: Relu reduction for fast private inference. In *International Conference on Machine Learning*. PMLR, 4839–4849.
- [92] Nandan Kumar Jha and Brandon Reagen. 2024. AERO: Softmax-Only LLMs for Efficient Private Inference. *arXiv preprint arXiv:2410.13060* (2024).
- [93] Nandan Kumar Jha and Brandon Reagen. 2024. DeepReShape: Redesigning Neural Networks for Efficient Private Inference. *arXiv:2304.10593* [cs.CR] <https://arxiv.org/abs/2304.10593>
- [94] Xiaoqian Jiang, Miran Kim, Kristin Lauter, and Yongsoo Song. 2018. Secure outsourced matrix computation and application to neural networks. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 1209–1222.
- [95] Dian Jiao, Xianglong Deng, Zhiwei Wang, Shengyu Fan, Yi Chen, Dan Meng, Rui Hou, and Mingzhe Zhang. 2025. Neo: Towards Efficient Fully Homomorphic Encryption Acceleration using Tensor Core. In *Proceedings of the 52nd ACM/IEEE International Symposium on Computer Architecture (ISCA)*. IEEE, Tokyo, Japan, 1–14.
- [96] Jae Hyung Ju, Jaiyoung Park, Jongmin Kim, Minsik Kang, Donghwan Kim, Jung Hee Cheon, and Jung Ho Ahn. 2024. NeuJeans: Private Neural Network Inference with Joint Optimization of Convolution and FHE Bootstrapping. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 4361–4375.
- [97] Wonkyung Jung, Sangpyo Kim, Jung Ho Ahn, Jung Hee Cheon, and Younho Lee. 2021. Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with GPUs. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2021), 114–148.
- [98] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. 2018. {GAZELLE}: A low latency framework for secure neural network inference. In *27th USENIX security symposium (USENIX security 18)*. 1651–1669.
- [99] Marcel Keller. 2020. MP-SPDZ: A Versatile Framework for Multi-Party Computation. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. doi:10.1145/3372297.3417872
- [100] Marcel Keller and Ke Sun. 2022. Secure quantized training for deep learning. In *International Conference on Machine Learning*. PMLR, 10912–10938.
- [101] Andrey Kim, Antonis Papadimitriou, and Yuriy Polyakov. 2022. Approximate homomorphic encryption with reduced approximation error. In *Cryptographers' Track at the RSA Conference*. Springer, 120–144.
- [102] Dongwoo Kim and Cyril Guyot. 2023. Optimized privacy-preserving cnn inference with fully homomorphic encryption. *IEEE Transactions on Information Forensics and Security* 18 (2023), 2175–2187.
- [103] Jongmin Kim, Wonseok Choi, and Jung Ho Ahn. 2024. Cheddar: A swift fully homomorphic encryption library for cuda gpus. *arXiv preprint arXiv:2407.13055* (2024).
- [104] Jongmin Kim, Sungmin Yun, Hyesung Ji, Wonseok Choi, Sangpyo Kim, and Jung Ho Ahn. 2025. Anaheim: Architecture and Algorithms for Processing Fully Homomorphic Encryption in Memory. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1158–1173.
- [105] Miran Kim, Xiaoqian Jiang, Kristin Lauter, Elkhan Ismayilzada, and Shayan Shams. 2022. Secure human action recognition by encrypted neural network inference. *Nature communications* 13, 1 (2022), 4799.
- [106] Miran Kim, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. 2023. Accelerating HE operations from key decomposition technique. In *Annual International Cryptology Conference*. Springer, 70–92.

- [107] Brian Knott, Shobha Venkataraman, Awni Hannun, Shubho Sengupta, Mark Ibrahim, and Laurens van der Maaten. 2021. Crypten: Secure multi-party computation meets machine learning. *Advances in Neural Information Processing Systems* 34 (2021), 4961–4973.
- [108] Nishat Koti, Mahak Pancholi, Arpita Patra, and Ajith Suresh. 2021. {SWIFT}: Super-fast and robust {Privacy-Preserving} machine learning. In *30th USENIX Security Symposium (USENIX Security 21)*. 2651–2668.
- [109] Nishat Koti, Arpita Patra, Rahul Rachuri, and Ajith Suresh. 2021. Tetrad: Actively secure 4pc for secure training and inference. *arXiv preprint arXiv:2106.02850* (2021).
- [110] Aleksandar Krastev, Nikola Samardzic, Simon Langowski, Srinivas Devadas, and Daniel Sanchez. 2024. A tensor compiler with automatic data packing for simple and efficient fully homomorphic encryption. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 126–150.
- [111] Souvik Kundu, Shunlin Lu, Yuke Zhang, Jacqueline Liu, and Peter A Beerel. 2023. Learning to linearize deep neural networks for secure and efficient private inference. *arXiv preprint arXiv:2301.09254* (2023).
- [112] Eunsang Lee, Joon-Woo Lee, Young-Sik Kim, and Jong-Seon No. 2022. Optimization of homomorphic comparison algorithm on rns-ckks scheme. *IEEE Access* 10 (2022), 26163–26176.
- [113] Eunsang Lee, Joon-Woo Lee, Junghyun Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and Woosuk Choi. 2022. Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions. In *International Conference on Machine Learning*. PMLR, 12403–12422.
- [114] Eunsang Lee, Joon-Woo Lee, Jong-Seon No, and Young-Sik Kim. 2021. Minimax approximation of sign function by composite polynomial for homomorphic comparison. *IEEE Transactions on Dependable and Secure Computing* 19, 6 (2021), 3711–3727.
- [115] Junghyun Lee, Eunsang Lee, Young-Sik Kim, Yongwoo Lee, Joon-Woo Lee, Yongjune Kim, and Jong-Seon No. 2023. Optimizing layerwise polynomial approximation for efficient private inference on fully homomorphic encryption: a dynamic programming approach. *arXiv preprint arXiv:2310.10349* (2023).
- [116] Joon-Woo Lee, HyungChul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, et al. 2022. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *IEEE Access* 10 (2022), 30039–30054.
- [117] Yongwoo Lee, Seonyoung Cheon, Dongkwan Kim, Dongyoon Lee, and Hanjun Kim. 2023. {ELASM}:{Error-Latency-Aware} Scale Management for Fully Homomorphic Encryption. In *32nd USENIX Security Symposium (USENIX Security 23)*. 4697–4714.
- [118] Yongwoo Lee, Seonyoung Heo, Seonyoung Cheon, Shinnung Jeong, Changsu Kim, Eunkyung Kim, Dongyoon Lee, and Hanjun Kim. 2022. HECATE: Performance-aware scale optimization for homomorphic encryption compiler. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 193–204.
- [119] Bo-hu Li, Bao-cun Hou, Wen-tao Yu, Xiao-bing Lu, and Chun-wei Yang. 2017. Applications of artificial intelligence in intelligent manufacturing: a review. *Frontiers of Information Technology & Electronic Engineering* 18, 1 (2017), 86–96.
- [120] Dacheng Li, Rulin Shao, Hongyi Wang, Han Guo, Eric P Xing, and Hao Zhang. 2022. Mpcformer: fast, performant and private transformer inference with mpc. *arXiv preprint arXiv:2211.01452* (2022).
- [121] Fabing Li, Yuanhao Zhai, Shuangyu Cai, and Mingyu Gao. [n. d.]. Seesaw: Compensating for Nonlinear Reduction with Linear Computations for Private Inference. In *Forty-first International Conference on Machine Learning*.
- [122] Long Li, Jianxin Lai, Peng Yuan, Tianxiang Sui, Yan Liu, Qing Zhu, Xiaojing Zhang, Linjie Xiao, Wenguang Chen, and Jingling Xue. 2025. ANT-ACE: An FHE Compiler Framework for Automating Neural Network Inference. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*. 193–208.
- [123] Qirui Li and Rui Zong. 2025. CAT: A GPU-Accelerated FHE Framework with Its Application to High-Precision Private Dataset Query. *arXiv preprint arXiv:2503.22227* (2025).
- [124] Yang Li, Xinyu Zhou, Yitong Wang, Liangxin Qian, and Jun Zhao. 2024. A Survey on Private Transformer Inference. *arXiv preprint arXiv:2412.08145* (2024).
- [125] Chenqi Lin, Tianshi Xu, Zebin Yang, Runsheng Wang, Ru Huang, and Meng Li. 2024. FastQuery: Communication-efficient Embedding Table Query for Private LLMs inference. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [126] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [127] Jian Liu, Mika Juuti, Yao Lu, and Nadarajah Asokan. 2017. Oblivious neural network predictions via minionn transformations. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 619–631.
- [128] Siqi Liu, Zhusen Liu, Donglong Chen, Wangchen Dai, Lu Zhou, Zhe Liu, Ray CC Cheung, and Çetin Kaya Koç. 2025. MLFormer: a high performance MPC linear inference framework for transformers. *Journal of Cryptographic Engineering* 15, 1 (2025), 2.
- [129] Xuanqi Liu and Zhuotao Liu. 2023. LLMs can understand encrypted prompt: Towards privacy-computing friendly transformers. *arXiv preprint arXiv:2305.18396* (2023).
- [130] Yan Liu, Jianxin Lai, Long Li, Tianxiang Sui, Linjie Xiao, Peng Yuan, Xiaojing Zhang, Qing Zhu, Wenguang Chen, and Jingling Xue. 2025. ReSBM: Region-based Scale and Minimal-Level Bootstrapping Management for FHE via Min-Cut. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 924–939.
- [131] Yanxin Liu and Qianqian Su. 2024. PPTIF: Privacy-Preserving Transformer Inference Framework for Language Translation. *IEEE Access* (2024).

- [132] Zeyu Liu and Yunhao Wang. 2023. Amortized functional bootstrapping in less than 7 ms, with $O^*(1)$ polynomial multiplications. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 101–132.
- [133] Zeyu Liu and Yunhao Wang. 2025. Relaxed functional bootstrapping: A new perspective on BGV/BFV bootstrapping. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 208–240.
- [134] Qian Lou, Yilin Shen, Hongxia Jin, and Lei Jiang. 2021. Safenet: A secure, accurate and fast neural network inference. In *International Conference on Learning Representations*.
- [135] Wen-jie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Cheng Hong, Kui Ren, Tao Wei, and WenGuang Chen. 2023. Bumblebee: Secure two-party inference framework for large transformers. *Cryptology ePrint Archive* (2023).
- [136] Jinglong Luo, Yehong Zhang, Zhuo Zhang, Jiaqi Zhang, Xin Mu, Hui Wang, Yue Yu, and Zenglin Xu. 2024. SecFormer: Fast and Accurate Privacy-Preserving Inference for Transformer Models via SMPC. In *Findings of the Association for Computational Linguistics ACL 2024*. 13333–13348.
- [137] Yukui Luo, Nuo Xu, Hongwu Peng, Chenghong Wang, Shijin Duan, Kaleel Mahmood, Wujie Wen, Caiwen Ding, and Xiaolin Xu. 2023. Aq2pnn: Enabling two-party privacy-preserving deep neural network inference with adaptive quantization. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 628–640.
- [138] Junming Ma, Yancheng Zheng, Jun Feng, Derun Zhao, Haoqi Wu, Wenjing Fang, Jin Tan, Chaofan Yu, Benyu Zhang, and Lei Wang. 2023. {SecretFlow-SPU}: A Performant and {User-Friendly} Framework for {Privacy-Preserving} Machine Learning. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 17–33.
- [139] Raghav Malik, Kabir Sheth, and Milind Kulkarni. 2023. Coyote: A compiler for vectorizing encrypted arithmetic circuits. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 118–133.
- [140] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. 2020. Delphi: A cryptographic inference system for neural networks. In *Proceedings of the 2020 Workshop on Privacy-Preserving Machine Learning in Practice*. 27–30.
- [141] Payman Mohassel and Peter Rindal. 2018. ABY3: A mixed protocol framework for machine learning. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 35–52.
- [142] Payman Mohassel and Yupeng Zhang. 2017. Secureml: A system for scalable privacy-preserving machine learning. In *2017 IEEE symposium on security and privacy (SP)*. IEEE, 19–38.
- [143] Lucien KL Ng and Sherman SM Chow. 2023. Sok: Cryptographic neural-network computation. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 497–514.
- [144] Isaac Kofi Nti, Adebayo Felix Adekoya, Benjamin Asubam Weyori, and Owusu Nyarko-Boateng. 2022. Applications of artificial intelligence in engineering and manufacturing: a systematic review. *Journal of Intelligent Manufacturing* 33, 6 (2022), 1581–1601.
- [145] OpenAI. 2022. ChatGPT. <https://openai.com/blog/chatgpt>.
- [146] Ali Şah Özcan, Can Ayduman, Enes Recep Türkoğlu, and Erkan Savaş. 2023. Homomorphic encryption on GPU. *IEEE Access* 11 (2023), 84168–84186.
- [147] Marie Paindavoine and Bastien Vialla. 2015. Minimizing the number of bootstrappings in fully homomorphic encryption. In *International Conference on Selected Areas in Cryptography*. Springer, 25–43.
- [148] Qi Pang, Jinhao Zhu, Helen Möllering, Wenting Zheng, and Thomas Schneider. 2024. Bolt: Privacy-preserving, accurate and efficient inference for transformers. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4753–4771.
- [149] Dongjin Park, Eunsang Lee, and Joon-Woo Lee. 2025. Powerformer: Efficient privacy-preserving transformer with batch rectifier-power max function and optimized homomorphic attention. *ACL* (2025).
- [150] Jaiyoung Park, Donghwan Kim, Jongmin Kim, Sangpyo Kim, Wonkyung Jung, Jung Hee Cheon, and Jung Ho Ahn. 2023. Toward practical privacy-preserving convolutional neural networks exploiting fully homomorphic encryption. *arXiv preprint arXiv:2310.16530* (2023).
- [151] Jaiyoung Park, Michael Jaemin Kim, Wonkyung Jung, and Jung Ho Ahn. 2022. AESPA: Accuracy preserving low-degree polynomial activation for fast private inference. *arXiv preprint arXiv:2201.06699* (2022).
- [152] Hongwu Peng, Shaoyi Huang, Tong Zhou, Yukui Luo, Chenghong Wang, Zigeng Wang, Jiahui Zhao, Xi Xie, Ang Li, Tony Geng, et al. 2023. Autorep: Automatic relu replacement for fast private network inference. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 5178–5188.
- [153] Hongwu Peng, Shanglin Zhou, Yukui Luo, Nuo Xu, Shijin Duan, Ran Ran, Jiahui Zhao, Shaoyi Huang, Xi Xie, Chenghong Wang, Tong Geng, Wujie Wen, Xiaolin Xu, and Caiwen Ding. 2023. RRNet: Towards ReLU-Reduced Neural Network for Two-party Computation Based Private Inference. *arXiv:2302.02292 [cs.CR]* <https://arxiv.org/abs/2302.02292>
- [154] Hongwu Peng, Shanglin Zhou, Yukui Luo, Nuo Xu, Shijin Duan, Ran Ran, Jiahui Zhao, Chenghong Wang, Tong Geng, Wujie Wen, et al. 2023. PASNet: polynomial architecture search framework for two-party computation-based secure neural network deployment. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [155] Robert Podschwadt and Daniel Takabi. 2020. Classification of Encrypted Word Embeddings using Recurrent Neural Networks.. In *PrivateNLP@ WSDM*. 27–31.
- [156] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [157] Deevashwer Rathee, Anwesh Bhattacharya, Rahul Sharma, Divya Gupta, Nishanth Chandran, and Aseem Rastogi. 2022. Secfloat: Accurate floating-point meets secure 2-party computation. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 576–595.

- [158] Deevashwer Rathee, Dacheng Li, Ion Stoica, Hao Zhang, and Raluca Popa. 2024. MPC-Minimized Secure LLM Inference. *arXiv preprint arXiv:2408.03561* (2024).
- [159] Deevashwer Rathee, Mayank Rathee, Rahul Kranti Kiran Goli, Divya Gupta, Rahul Sharma, Nishanth Chandran, and Aseem Rastogi. 2021. Sirnn: A math library for secure rnn inference. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1003–1020.
- [160] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. 2020. Cryptflow2: Practical 2-party secure inference. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 325–342.
- [161] M Sadegh Riazi, Mohammad Samragh, Hao Chen, Kim Laine, Kristin Lauter, and Farinaz Koushanfar. 2019. {XONN}:{XNOR-based} oblivious deep neural network inference. In *28th USENIX Security Symposium (USENIX Security 19)*. 1501–1518.
- [162] Lorenzo Rovida and Alberto Leporati. 2024. Encrypted image classification with low memory footprint using fully homomorphic encryption. *Cryptology ePrint Archive* (2024).
- [163] SEAL 2023. Microsoft SEAL (release 4.1). <https://github.com/Microsoft/SEAL>. Microsoft Research, Redmond, WA..
- [164] Samad Sepasgozar, Reyhaneh Karimi, Leila Farahzadi, Farimah Moezzi, Sara Shirowzhan, Sane M. Ebrahimzadeh, Felix Hui, and Lu Aye. 2020. A systematic content review of artificial intelligence and the internet of things applications in smart home. *Applied Sciences* 10, 9 (2020), 3074.
- [165] Liyan Shen, Ye Dong, Binxing Fang, Jinqiao Shi, Xuebin Wang, Shengli Pan, and Ruisheng Shi. 2022. ABNN2: secure two-party arbitrary-bitwidth quantized neural network predictions. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 361–366.
- [166] Shiyu Shen, Hao Yang, Yu Liu, Zhe Liu, and Yunlei Zhao. 2022. CARM: CUDA-accelerated RNS multiplication in word-wise homomorphic encryption schemes for internet of things. *IEEE Trans. Comput.* 72, 7 (2022), 1999–2010.
- [167] Shiyu Shen, Hao Yang, Zhe Liu, Ying Liu, Xianhui Lu, Wangchen Dai, Lu Zhou, Yunlei Zhao, and Ray CC Cheung. 2025. VeloFHE: GPU Acceleration for FHEW and TFHE Bootstrapping. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2025, 3 (2025), 81–114.
- [168] Kaustubh Shivdikar, Yuhui Bao, Rashmi Agrawal, Michael Shen, Gilbert Jonatan, Evelio Mora, Alexander Ingare, Neal Livesay, José L. Abellán, John Kim, et al. 2023. Gme: Gpu-based microarchitectural extensions to accelerate homomorphic encryption. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 670–684.
- [169] Andrei Stoian, Jordan Frery, Roman Bredehoft, Luis Montero, Celia Kherfallah, and Benoit Chevallier-Mames. 2023. Deep neural networks for encrypted inference with tfhe. In *International Symposium on Cyber Security, Cryptology, and Machine Learning*. Springer, 493–500.
- [170] AVLN Sujith, Guna Sekhar Sajja, V Mahalakshmi, Shibili Nuhmani, and Balaji Prasanalakshmi. 2022. Systematic review of smart health monitoring using deep learning and Artificial intelligence. *Neuroscience Informatics* 2, 3 (2022), 100028.
- [171] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [172] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).
- [173] Vertex.AI and Intel AI Lab. 2024. PlaidML: PlaidML - a framework for making deep learning work everywhere. <https://github.com/plaidml/plaidml>. Accessed: 2025-06-04.
- [174] Alexander Vian, Patrick Jattke, Miro Haller, and Anwar Hithnawi. 2023. {HECO}: Fully homomorphic encryption compiler. In *32nd USENIX Security Symposium (USENIX Security 23)*. 4715–4732.
- [175] Sameer Wagh. 2022. Pika: Secure computation using function secret sharing over rings. *Proceedings on Privacy Enhancing Technologies* (2022).
- [176] Sameer Wagh, Shruti Tople, Fabrice Benhamouda, Eyal Kushilevitz, Prateek Mittal, and Tal Rabin. 2020. Falcon: Honest-majority maliciously secure framework for private deep learning. *arXiv preprint arXiv:2004.02229* (2020).
- [177] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. 2016. EMP-toolkit: Efficient MultiParty computation toolkit. <https://github.com/emp-toolkit>.
- [178] Yongqin Wang, G Edward Suh, Wenjie Xiong, Benjamin Lefaudeaux, Brian Knott, Murali Annavaram, and Hsien-Hsin S Lee. 2022. Characterization of mpc-based private inference for transformer-based models. In *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 187–197.
- [179] Zhiwei Wang, Peinan Li, Rui Hou, Zhihao Li, Jiangfeng Cao, XiaoFeng Wang, and Dan Meng. 2023. He-booster: An efficient polynomial arithmetic acceleration on gpus for fully homomorphic encryption. *IEEE Transactions on Parallel and Distributed Systems* 34, 4 (2023), 1067–1081.
- [180] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [181] Haoqi Wu, Wenjing Fang, Yancheng Zheng, Junming Ma, Jin Tan, Yinggui Wang, and Lei Wang. 2024. Ditto: Quantization-aware Secure Inference of Transformers upon MPC. *arXiv preprint arXiv:2405.05525* (2024).
- [182] Yu Xiao, Feng-Hao Liu, Yu-Te Ku, Ming-Chien Ho, Chih-Fan Hsu, Ming-Ching Chang, Shih-Hao Hung, and Wei-Chao Chen. 2025. GPU Acceleration for FHEW/TFHE Bootstrapping. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2025, 1 (2025), 314–339.
- [183] Tianshi Xu, Meng Li, and Runsheng Wang. 2024. HEQuant: Marrying Homomorphic Encryption and Quantization for Communication-Efficient Private Inference. *arXiv preprint arXiv:2401.15970* (2024).
- [184] Tianshi Xu, Meng Li, Runsheng Wang, and Ru Huang. 2023. Falcon: Accelerating homomorphically encrypted convolutions for efficient private mobile network inference. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [185] Tianshi Xu, Wen-jie Lu, Jiangrui Yu, Yi Chen, Chenqi Lin, Runsheng Wang, and Meng Li. 2025. Breaking the Layer Barrier: Remodeling Private Transformer Inference with Hybrid CKKS and MPC. *USENIX Security Symposium* (2025).
- [186] Tianshi Xu, Lemeng Wu, Runsheng Wang, and Meng Li. 2024. PrivCirNet: Efficient Private Inference via Block Circulant Transformation. *arXiv preprint arXiv:2405.14569* (2024).

- [187] Tianshi Xu, Shuzhang Zhong, Wenxuan Zeng, Runsheng Wang, and Meng Li. 2024. PrivQuant: Communication-Efficient Private Inference with Quantized Network/Protocol Co-Optimization. *arXiv preprint arXiv:2410.09531* (2024).
- [188] Guang Yan, Yuhui Zhang, Zimu Guo, Lutan Zhao, Xiaojun Chen, Chen Wang, Wenhao Wang, Dan Meng, and Rui Hou. 2025. Comet: Accelerating Private Inference for Large Language Model by Predicting Activation Sparsity. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2827–2845.
- [189] Hao Yang, Shiyu Shen, Wangchen Dai, Lu Zhou, Zhe Liu, and Yunlei Zhao. 2024. Phantom: A cuda-accelerated word-wise homomorphic encryption library. *IEEE Transactions on Dependable and Secure Computing* 21, 5 (2024), 4895–4906.
- [190] AC Yao. [n. d.]. How to generate and exchange secrets (extended abstract). FOCS.
- [191] Jiangrui Yu, Wenxuan Zeng, Tianshi Xu, Renze Chen, Yun Liang, Runsheng Wang, Ru Huang, and Meng Li. 2024. FlexHE: A flexible Kernel Generation Framework for Homomorphic Encryption-Based Private Inference. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [192] Ardhi Wiratama Baskara Yudha, Jiaqi Xue, Qian Lou, Huiyang Zhou, and Yan Solihin. 2024. BoostCom: Towards Efficient Universal Fully Homomorphic Encryption by Boosting the Word-wise Comparisons. In *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques*. 121–132.
- [193] Wenxuan Zeng, Ye Dong, Jinjin Zhou, Junming Ma, Jin Tan, Runsheng Wang, and Meng Li. 2025. MPCache: MPC-Friendly KV Cache Eviction for Efficient Private Large Language Model Inference. *arXiv preprint arXiv:2501.06807* (2025).
- [194] Wenxuan Zeng, Meng Li, Wenjie Xiong, Tong Tong, Wen-jie Lu, Jin Tan, Runsheng Wang, and Ru Huang. 2023. Mpcvit: Searching for accurate and efficient mpc-friendly vision transformer with heterogeneous attention. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 5052–5063.
- [195] Wenxuan Zeng, Meng Li, Haichuan Yang, Wen-jie Lu, Runsheng Wang, and Ru Huang. 2023. CoPriv: network/protocol co-optimization for communication-efficient private inference. *Advances in Neural Information Processing Systems* 36 (2023), 78906–78925.
- [196] Wenxuan Zeng, Tianshi Xu, Meng Li, and Runsheng Wang. 2024. EQO: Exploring Ultra-Efficient Private Inference with Winograd-Based Protocol and Quantization Co-Optimization. *arXiv preprint arXiv:2404.09404* (2024).
- [197] Jiawen Zhang, Xinpeng Yang, Lipeng He, Kejia Chen, Wen-jie Lu, Yinghao Wang, Xiaoyang Hou, Jian Liu, Kui Ren, and Xiaohu Yang. 2024. Secure transformer inference made non-interactive. *Cryptology ePrint Archive* (2024).
- [198] Xinqiao Zhang, Mohammad Samragh, Siam Hussain, Ke Huang, and Farinaz Koushanfar. 2024. Scalable Binary Neural Network applications in Oblivious Inference. *ACM Transactions on Embedded Computing Systems* 23, 3 (2024), 1–18.
- [199] Yuke Zhang, Dake Chen, Souvik Kundu, Chenghao Li, and Peter A Beerel. 2023. Sal-vit: Towards latency efficient private inference on vit using selective attention search with a learnable softmax approximation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 5116–5125.
- [200] Yancheng Zhang, Jiaqi Xue, Mengxin Zheng, Mimi Xie, Mingzhe Zhang, Lei Jiang, and Qian Lou. 2025. Cipherprune: Efficient and scalable private transformer inference. *arXiv preprint arXiv:2502.16782* (2025).
- [201] Yancheng Zhang, Mengxin Zheng, Yuzhang Shang, Xun Chen, and Qian Lou. [n. d.]. HEPrune: Fast Private Training of Deep Neural Networks With Encrypted Data Pruning. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- [202] Itamar Zimmerman, Allon Adir, Ehud Aharoni, Matan Avitan, Moran Baruch, Nir Drucker, Jenny Lerner, Ramy Masalha, Reut Meiri, and Omri Soceanu. 2024. Power-Softmax: Towards Secure LLM Inference over Encrypted Data. *arXiv preprint arXiv:2410.09457* (2024).
- [203] Itamar Zimmerman, Moran Baruch, Nir Drucker, Gilad Ezov, Omri Soceanu, and Lior Wolf. 2023. Converting transformers to polynomial form for secure inference over homomorphic encryption. *arXiv preprint arXiv:2311.08610* (2023).